

Long Context, Retrieval, Structured Indexing, and Search Agents:

An Educational Architecture Review Centered on Progressive Retrieval Attention

Jorge Simão

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity
Tutorial and research-design draft

August 2026

Abstract

Language models can gain access to more information by making the token window larger, restricting attention, carrying recurrent state, consulting an external memory, retrieving text before generation, retrieving latent key/value representations during attention, navigating a structured document index, or invoking search tools iteratively. These mechanisms are often discussed as separate subfields even though they answer the same architectural questions: *what is stored, how is it named, when is relevance decided, where is retrieved information materialized, and how can its causal contribution be tested?*

This tutorial organizes the field into seven categories and uses Progressive Retrieval Attention (PRAAttention, PRA) as a unifying reference point. PRA represents external context through explicit URI-like handles, resolves selected objects into layer-specific key/value memory, and permits demand-driven access through a dedicated attention branch. We first reconstruct the PRA position and standalone prototype papers, including their preliminary WikiText-2 results and counterfactual evaluation doctrine. We then explain representative approaches using motivation, formalism, pseudocode, PyTorch sketches, diagrams, original experiments, research lineage, downstream influence, relation to PRA, and concrete comparison experiments. The final sections develop a common design space, propose fair experimental matrices, discuss provenance and the 2026 PaperTrail interface, and derive extensions that combine hierarchical references, learned routing, cache reuse, and agentic escalation. The intended audience is students, researchers, and implementers who need both a conceptual map and reusable experimental designs.

Contents

1	Introduction	4
1.1	Seven categories	4
1.2	A shared vocabulary	4
2	Progressive Retrieval Attention	4
2.1	The proposal reconstructed from Papers 0 and 1	4
2.2	Reference graph, resolver, and runtime state	5
2.3	Selection, recursive expansion, and the implemented specialization	5
2.4	Layer-specific materialization and PRAAttention	5
2.5	Reference-data construction and leakage controls	6
2.6	Complexity and the systems claim	6
2.7	What the preliminary experiments establish—and do not establish	7
2.8	Counterfactual doctrine and falsifiable comparison	7
3	Category I: Dense, Sparse, and Approximate Long-Context Attention	8
3.1	Dense attention and positional extension	8
3.2	Longformer	8

3.3	BigBird	9
3.4	Performer	10
4	Category II: Recurrence and Compression	10
4.1	Transformer-XL	10
4.2	Compressive Transformer	11
4.3	Recurrent Memory Transformer (RMT)	11
4.4	Infini-attention	12
5	Category III: Differentiable and Episodic External Memory	12
5.1	Memory Networks	13
5.2	Differentiable Neural Computer (DNC)	13
5.3	Memformer	14
5.4	Memorizing Transformer	14
6	Category IV: Retrieval-Augmented Pretraining and Generation	15
6.1	kNN-LM	15
6.2	REALM	15
6.3	Retrieval-Augmented Generation (RAG)	16
6.4	RETRO	16
7	Category V: Attention-Level Retrieval and KV Runtime Management	17
7.1	Unlimiformer	17
7.2	PagedAttention	18
7.3	StreamingLLM	18
7.4	H ₂ O: Heavy-Hitter Oracle	19
7.5	Scissorhands	19
7.6	SnapKV	20
8	Category VI: Structural and Hierarchical Indexing	20
8.1	Hierarchical Attention Networks (HAN)	20
8.2	RAPTOR	21
8.3	MemWalker	21
8.4	PageIndex	22
8.5	GraphRAG	22
8.6	PaperTrail as provenance and HCI	23
9	Category VII: Agents with Search and Tool Use	23
9.1	ReAct	23
9.2	Self-Ask with Search	24
9.3	IRCoT	24
9.4	Toolformer	25
9.5	WebGPT and browser agents	25
9.6	Recursive Language Models and context as environment	26
9.7	A tiered agent-PRA design	27
10	One-Page Architecture Landscape	28
11	Cross-Architecture Comparison	29
12	Adapting Original Experiments to PRA	29

12.1 A benchmark matrix	29
12.2 Common conditions	29
12.3 PRA-specific causal matrix	30
12.4 Metrics beyond task accuracy	30
13 Research Lineage and Influence	30
14 Design Extensions Derived from the Survey	30
14.1 Hierarchical Progressive Retrieval Attention	30
14.2 Agent-distilled routing	30
14.3 Multi-fidelity object caches	30
14.4 Reference compilation	31
14.5 Causal provenance graph	31
15 Discussion	31
15.1 The real design choice is the access contract	31
15.2 When PRA should not win	31
15.3 When PRA has a plausible advantage	31
15.4 Training difficulty	31
15.5 Object identity versus semantic similarity	31
15.6 Impact on agents	31
16 Conclusion	32

1 Introduction

The canonical Transformer converts a sequence into contextual representations using dense self-attention. Its flexibility comes with a quadratic interaction matrix and an implicit assumption: all potentially useful information is already present as a flat token sequence. Long-document and knowledge-intensive tasks violate that assumption in several distinct ways. The information may be too large to fit; relevance may be sparse; the useful evidence may be hierarchically organized; a query may evolve during reasoning; or the relevant source may not be known until generation has begun.

The literature responds with mechanisms at different system boundaries. Sparse attention changes the graph within a prompt. Recurrence carries state across prompt segments. Differentiable memory compresses or stores prior activations. Retrieval-augmented generation chooses passages and inserts them as text. Retrieval-attention systems index hidden states or key/value vectors. Structural indexes let a model navigate sections and summaries. Search agents act outside the model through repeated queries and tool calls. PRA is best understood not as replacing all of these, but as proposing a different interface between them: lightweight references remain in the local stream; named external objects are resolved only when useful; and the materialized representation is a layer-local latent memory rather than necessarily prompt text.

1.1 Seven categories

1. **Dense, sparse, and approximate long-context attention:** extend the visible token graph.
2. **Recurrence and compression:** carry or summarize state across segments.
3. **Differentiable external memory:** read and write learned memory slots or episodic stores.
4. **Retrieval-augmented pretraining and generation:** retrieve textual evidence before or during generation.
5. **Attention-level and KV retrieval:** retrieve hidden states or key/value memories directly into attention.
6. **Structural and hierarchical indexing:** navigate document, graph, or summary structure rather than flat chunks.
7. **Agents with search tools:** iteratively formulate queries, inspect results, follow links, and decide when to stop.

1.2 A shared vocabulary

Let $x_{1:T}$ be local tokens, q_t a query representation at generation step t , $\mathcal{S}$ an external store, $R(q_t, \mathcal{S})$ a selection operator, and M_t the materialized memory. A general access architecture is

$$C_t = R(q_t, \mathcal{S}; \pi), \tag{1}$$

$$M_t = \text{materialize}(C_t; \rho), \tag{2}$$

$$h'_t = \text{combine}(h_t, M_t; \gamma), \tag{3}$$

where π is a selection policy, ρ determines representation (tokens, summaries, hidden states, KV tensors, tool observations), and γ determines fusion. Every method in this review can be located by its choices for $\mathcal{S}$, R , ρ , and γ .

2 Progressive Retrieval Attention

2.1 The proposal reconstructed from Papers 0 and 1

PRA is easiest to understand as an *access contract*, not as a claim that one more attention formula solves long context. Paper 0 proposes a logical context graph whose nodes are stable, URI-addressed objects and whose edges are semantic anchors. Paper 1 specializes that proposal into a small decoder-only PyTorch model. The two manuscripts make five separations that must remain explicit throughout this survey:

1. **naming:** a handle states which object may be consulted;
2. **selection:** a policy chooses among advertised objects;
3. **resolution:** a resolver binds a URI, policy, and version to content;
4. **materialization:** content becomes text, a summary, hidden states, or layer-specific keys and values; and
5. **fusion:** the decoder consumes that representation through prompt tokens, cross-attention, native KV injection, or another transport.

This decomposition is pedagogically important. A better retriever does not establish a better transport, and a useful memory branch does not establish that the router selected the correct object. PRA’s distinctive package is explicit URI identity, separation of naming from materialization, reusable per-layer KV objects, progressive triggering, and counterfactual evaluation. Each component has precursors; the package, interface, and experimental doctrine are the proposal under study.

2.2 Reference graph, resolver, and runtime state

Let a context graph be $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. A node $v = (u, z, s, m)$ contains a stable URI u , content z , a cheap summary s , and metadata m . An edge (v_i, a, v_j) says that anchor a in v_i expands to v_j . The resolver is deliberately outside the neural layer:

$$\rho(u, p, t) \longrightarrow (z, s, \text{children}, m), \quad (4)$$

where p is an authorization domain and t identifies logical time or version. This interface permits a file, database row, document section, code symbol, graph query, or tool result to be named in the same logical space without pretending that they share the same physical representation.

The runtime state before token t is

$$\Omega_t = (\mathcal{G}, \mathcal{H}_t, A_t, \mathcal{C}_t, p_t, \nu_t), \quad (5)$$

with advertised handles $\mathcal{H}_t$, active references A_t , resident materializations $\mathcal{C}_t$, policy state p_t , and provenance/version state ν_t . A persistent cache key must include at least $(u, v, \ell, m, \theta, p)$: URI, content version, consumer layer, model identity, relevant encoder parameters, and policy domain. Paper 1 uses only the URI because it rebuilds and discards a private cache for every example; that specialization prevents cross-example leakage but does not demonstrate persistent reuse.



2.3 Selection, recursive expansion, and the implemented specialization

At layer ℓ , a query $q_{t,\ell}$ ranks the summaries of candidate objects:

$$\mathcal{S}_{k,\tau}(q, \mathcal{C}) = \text{TopK}_u \{ \cos(q, s_u) : \cos(q, s_u) \geq \tau \}. \quad (6)$$

An expansion operator $\mathcal{X}(A, \mathcal{G}, b, d)$ follows children subject to branching budget b and depth d . The intended transition is therefore

$$A_t = \mathcal{S}_{k,\tau}(q_t, \mathcal{C}_t), \quad A'_t = \mathcal{X}(A_t, \mathcal{G}, b, d), \quad (7)$$

$$\mathcal{C}'_t = \text{admit}(\text{encode}(\rho(A'_t))), \quad (y_t, \Omega_{t+1}) = \text{attend}(x_{\leq t}, \mathcal{C}'_t, \Omega_t). \quad (8)$$

Paper 1 implements a much narrower case: the last hidden state is the query, mean summary token embeddings are candidates, all one-to-five references are admitted in the reported reference experiment, recursive expansion is the identity, and caches are constructed before an example forward pass. Thus “progressive”, “recursive”, and “persistent” describe the research target, not measured properties of the current pilot.

2.4 Layer-specific materialization and PRAAttention

For URI u , the cache entry is

$$C_u = (u, z_u, s_u, \bar{s}_u, \{(K_{u,\ell}, V_{u,\ell})\}_{\ell \in \mathcal{L}_{\text{PRA}}}). \quad (9)$$

The reference text is passed through the same decoder with external memory disabled. For each PRA-enabled layer, the pre-attention normalized hidden states are projected by that layer’s W_K and W_V and detached. A selected set $\mathcal{U}$ is concatenated within the layer. Local attention remains causal; reference memory is a separate, non-causal cross-attention source:

$$O_{\text{local}} = \text{softmax}((QK^\top + M_{\text{causal}})/\sqrt{d_h})V, \quad (10)$$

$$O_{\text{mem}} = W_{O,\text{mem}} \text{ merge } \left[\text{softmax}(QK_{\mathcal{U},\ell}^\top/\sqrt{d_h})V_{\mathcal{U},\ell} \right], \quad (11)$$

$$O = O_{\text{local}} + \alpha O_{\text{mem}}. \quad (12)$$

The prototype zero-initializes $W_{O,\text{mem}}$ for isolated phase-2 adaptation. This makes the memory contribution exactly zero at initialization and preserves the local path when only that projection is trained. It is a useful engineering control, not an intrinsic no-forgetting theorem for PRA.

Listing 1: Reference-memory construction and consumption in Paper 1

```

for sample in batch: # private semantic address space
    cache = {}
    for handle in sample.references:
        obj = resolver.resolve(handle.uri, policy, version)
        h_ref = model.encode(obj.text, pra_enabled=False)
        entry = {"summary": mean_embed(obj.summary), "kv": {}}
        for layer in pra_layers:
            entry["kv"][layer.id] = layer.project_kv(h_ref[layer.id]).detach()
        cache[handle.uri] = entry
    logits = model(sample.tokens, pra_cache=cache)

```

Listing 2: Minimal executable shape of the PRA branch

```

class PRAAttention(nn.Module):
    def __init__(self, d, heads):
        super().__init__()
        self.local = nn.MultiheadAttention(d, heads, batch_first=True)
        self.memory = nn.MultiheadAttention(d, heads, batch_first=True)
        self.memory_out = nn.Linear(d, d)
        nn.init.zeros_(self.memory_out.weight); nn.init.zeros_(self.memory_out.bias)

    def forward(self, x, memory=None, causal_mask=None):
        local, _ = self.local(x, x, x, attn_mask=causal_mask,
                               need_weights=False)
        if memory is None or memory.size(1) == 0:
            return local
        mem, _ = self.memory(x, memory, memory, need_weights=False)
        return local + self.memory_out(mem)

```

2.5 Reference-data construction and leakage controls

The version-1 pilot split a WikiText-2 article into one-to-five URI-addressed earlier parts and a held-out continuation, but regenerated examples from each model seed and labeled every earlier part relevant. It is retained as engineering history, not causal selection evidence. Version 2 fixes data generation with seed 1729 and shares the same 512-example corpus across architectures and model seeds. It yields 409/51/52 train/validation/test examples, assigns candidate identifiers and order independently of relevance, designates one adjacent source part as the expected reference, and records source/split provenance. The candidate-count-weighted test chance rate is 0.4202.

Listing 3: Reproducible version-2 reference corpus construction

```

rng = Random(1729)
entries = parse_wikitext_articles(raw_text)
for article_id, entry in stable_order(entries):
    parts = split_natural_units(entry, min_parts=2, max_parts=6)
    target_i = choose_target_with_predecessor(parts, rng)
    positive = target_i - 1
    distractors = sample_other_prior_parts(parts, positive, rng)
    candidates = rng.shuffle([positive] + distractors)
    emit(prompt=make_handles(candidates), answer=parts[target_i],
         expected_uri=uri(article_id, positive),
         source_hash=sha256(entry), generator_seed=1729)
fixed_train, fixed_val, fixed_test = split_once(examples, 409, 51, 52)

```

Leakage controls are part of the architecture, not merely data hygiene. Candidate IDs must not encode relevance; all architectures must receive identical examples; train, validation, and test source partitions must be stable; reference content must be private to its example; and batched physical storage must preserve a sequence-to-cache mask. Content-only, random-URI, same-URI/wrong-version, shuffled, irrelevant, empty, oracle, and all-content-local variants are required to detect shortcuts. The adjacent-part label is still only a programmatic proxy; future data must mark human-verified necessary evidence.

2.6 Complexity and the systems claim

Let L be decoder layers, L_p PRA layers, T local tokens, r resolved references, m_i tokens in object i , and M selected resident memory tokens. Ignoring constant head factors, the stages cost

Stage	Asymptotic work	What must be measured
resolve encode	$O(r)$ lookups plus source I/O $O(Ld \sum_i m_i^2)$	authorization, version check, bytes and latency cold tokenization/encoding and accelerator transfer
cache projec- tion	$O(L_p d^2 \sum_i m_i)$	per-layer projection, bytes written, deduplication
route	$O(rd)$ exact summary scoring	candidate count, top- k latency, recall and calibration
cross-attend	$O(L_p T M d)$	selected tokens, layer utilization, synchronized latency
resident KV	$2wL_p M d$ bytes	allocator overhead, hit rate, eviction and tenant isolation

These terms expose the central unproven systems hypothesis. A shorter prompt is not an efficiency win if all references are re-encoded for every request. PRA becomes plausible when M is small, resolution avoids unnecessary objects, encoded KVs are reused across tokens or authorized requests, or quality improves at a fixed total cost. Results must therefore report cold, warm, and shared-cache regimes and separate lookup hits from *avoided encoding work*.

2.7 What the preliminary experiments establish—and do not establish

All models have four layers, four heads, width 256, context 128, dropout 0.1, and a 2,000-token BPE vocabulary. The initial two-seed pilot processes 524,288 tokens. The controlled follow-up uses seeds 1, 7, and 21, 1,048,576 ordinary-language tokens, fixed reference data, and 512 reference-stage optimizer steps. Parameter counts are 4,216,320 for same-width SelfAttention, 4,479,488 for full PRA, and 4,347,904 for the two-local/ two-PRA hybrid. FFN-expanded controls contain 4,478,976 and 4,347,648 parameters.

Table 1: Controlled three-seed ordinary-language results transcribed from Paper 1. Population standard deviations are across seeds 1, 7, and 21.

Model	Parameters	WikiText-2 test loss
SelfAttention, same width	4,216,320	4.6378 ± 0.0431
SelfAttention, matched to full PRA	4,478,976	4.5706 ± 0.0258
Full PRA	4,479,488	4.5463 ± 0.0270
SelfAttention, matched to hybrid	4,347,648	4.5725 ± 0.0362
Hybrid PRA	4,347,904	4.6049 ± 0.0359

Full PRA is 0.0243 loss units below its nearly matched control, while the hybrid is 0.0323 above its matched control. This supports compatibility and approximate parity at this scale, not architectural superiority. In the fixed reference task, valid-reference loss after frozen-path training is 4.7052 for full PRA and 4.7932 for hybrid PRA. Complete disabling raises loss by 0.0292 and 0.0214, but shuffled and irrelevant penalties are only 0.0064–0.0088 and oracle content changes loss by at most 0.0018. Top-1 selection remains near or below chance. The branch contributes, but correct content has not been shown to cause the gain. The current experiment therefore passes compatibility and preservation gates and fails the causal-selection gate.

The source manuscript reports aggregate seed means and population standard deviations, not a complete seed-by-seed ledger for every run. A submission version must export one row per run (architecture, seed, data hash, checkpoint hash, trainable parameters, tokens, loss, uncertainty inputs, timing, and counterfactual condition) rather than attempting to reconstruct individual values from aggregates.

2.8 Counterfactual doctrine and falsifiable comparison

For loss L , define

$$\Delta_{\text{use}} = L_{\text{disabled}} - L_{\text{valid}}, \quad \Delta_{\text{content}} = L_{\text{shuffled}} - L_{\text{valid}}. \quad (13)$$

$\Delta_{\text{use}} > 0$ only shows that activating the path helps. A credible content claim requires a reproducible $\Delta_{\text{content}} > 0$, a similar penalty for irrelevant references, increasing advantage on locally insufficient examples, and routing above a candidate-count-aware chance baseline. Attention weights are diagnostics; removal, substitution, stale-version, and mediation-style interventions supply stronger evidence.

The decisive experiment crosses selection and transport: oracle versus learned selection; prompt-text, cross-attention, and native-KV transport; valid/disabled/shuffled/irrelevant/ oracle content; and cold/warm/shared caches. It

matches model parameters, trainable parameters, source corpus, visible and retrieved token budgets, optimizer tokens, FLOPs, peak memory, and synchronized latency. PRA’s efficiency thesis is falsified in a target regime if it does not improve the quality–cost frontier after reference encoding and cache misses are included, or if correct content does not outperform corrupted controls.

3 Category I: Dense, Sparse, and Approximate Long-Context Attention

This family changes the token-interaction graph after information has already been materialized as one sequence. It is therefore the cleanest “all evidence local” control for PRA: selection is either absent or encoded in the attention mask, object identity is reduced to position, and no cross-request materialization is implied.

3.1 Dense attention and positional extension

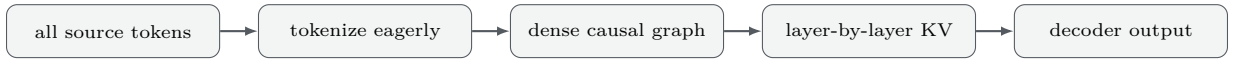
Motivation and mechanism. Dense causal attention is the least structured but most general access rule: every position can read every earlier position. RoPE scaling, position interpolation, and long-context continued pretraining extend the usable position range without changing this graph. For T tokens,

$$A = \text{softmax}((QK^\top + M_{\text{causal}})/\sqrt{d_h})V, \quad \text{time } O(T^2d), \text{ attention storage } O(T^2). \quad (14)$$

The important teaching point is that positional extrapolation and attention efficiency are different problems. A model may accept a long index while still allocating quadratic work, or execute efficiently while failing to use distant positions.

Listing 4: Dense causal attention; PyTorch uses optimized kernels when possible

```
def dense_causal(q, k, v): # [B,H,T,D]
    return F.scaled_dot_product_attention(q, k, v, is_causal=True)
```



Evidence, failure modes, and lineage. Standard language modeling and long-context adaptations use perplexity, long-document QA, retrieval probes, and throughput/memory sweeps. Dense attention is a strong baseline when sources are short or densely relevant, and modern fused kernels make it unexpectedly competitive at moderate lengths. Its failures include quadratic cost, distraction, positional degradation, and lack of stable object identity or reuse beyond exact prefixes. Needle tests diagnose reachability but do not establish synthesis or citation behavior.

Relation and comparison to PRA. Dense context performs eager materialization; PRA advertises a potentially larger logical context and materializes selected objects. Compare them on the same source corpus under (i) equal visible-plus-retrieved tokens, (ii) equal FLOPs, (iii) equal peak KV bytes, and (iv) equal synchronized latency. Sweep evidence density and object reuse. Dense attention should be expected to win when most tokens matter once; PRA’s hypothesis is strongest when few named objects serve many queries. The all-content-local condition is also PRA’s transport upper control.

3.2 Longformer

Motivation and mechanism. Longformer observes that document dependencies are mostly local but a few task positions need global connectivity. A token attends within a window and, optionally, to a set $\mathcal{G}$ of global positions [Beltagy et al., 2020]:

$$\mathcal{N}(i) = \{j : |i - j| \leq w/2\} \cup \mathcal{G}, \quad O(Tw + T|\mathcal{G}|). \quad (15)$$

Local layers propagate information gradually; global tokens create short paths across the document and can be task-selected (for example, question tokens in QA). Unlike truncation, the model retains an explicit sparse graph over the entire input.

Listing 5: Educational Longformer mask; production kernels avoid materializing T^2

```
def longformer_mask(T, window, global_pos, device):
    i = torch.arange(T, device=device)
    local = (i[:, None] - i[None, :]).abs() <= window // 2
    g = torch.zeros(T, dtype=torch.bool, device=device)
    g[global_pos] = True
    return local | g[:, None] | g[None, :]
```



Evidence and paper trail. The original paper studies character language modeling, long-document classification, and QA; its encoder-decoder continuation LED applies the pattern to long-input generation. Comparisons include RoBERTa-style dense baselines and task-specific long-document systems, using bits-per-character, accuracy/F1, and generation metrics. Longformer became a template for domain adaptations and local/global attention. Its main failure is structural: useful long-range routes must be known through the global mask or emerge through multiple local hops. Fixed windows can split tables, citations, or cross-section evidence, and global positions can become a bottleneck.

PRA relation and matched experiment. Longformer makes relevance a sparse edge decision over already materialized tokens; PRA makes it an object-admission decision before latent transport. Give both the same documents and vary evidence sparsity, distance, and whether section boundaries are known. Match the number of attended key positions: Longformer local/global edges versus PRA top- k object tokens. Add shuffled global markers and shuffled URI content. This separates sensitivity to graph placement from sensitivity to object semantics. A hybrid can use Longformer locally inside a large resolved object and PRA between objects.

3.3 BigBird

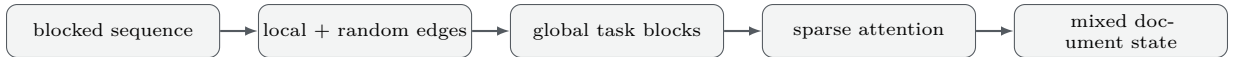
Motivation and mechanism. BigBird combines window, random, and global edges so that each token has bounded degree while the graph mixes rapidly [Zaheer et al., 2020]:

$$\mathcal{N}(i) = \mathcal{N}_{\text{window}}(i) \cup \mathcal{N}_{\text{random}}(i) \cup \mathcal{G}. \quad (16)$$

The random component supplies shortcuts without content-dependent routing; global tokens connect task anchors to all positions. Under the paper’s graph assumptions this pattern retains strong expressivity results while reducing sequence complexity to linear in T for fixed block sizes.

Listing 6: Block-level BigBird adjacency sketch

```
def bigbird_blocks(n_blocks, radius, random_k, global_blocks=(0,)):
    edges = []
    for i in range(n_blocks):
        local = range(max(0, i-radius), min(n_blocks, i+radius+1))
        random = torch.randperm(n_blocks)[:random_k].tolist()
        edges.append(sorted(set(local) | set(random) | set(global_blocks)))
    return edges
```



Evidence, ablations, and limitations. BigBird evaluates natural-language QA and summarization as well as genomic sequences, comparing dense BERT/RoBERTa-style models and other efficient Transformers with task accuracy/F1, ROUGE, and sequence-scale behavior. The meaningful ablation is the contribution of local, random, and global edges. Random connectivity is cheap and robust on average but can miss a particular necessary route; global tokens require a placement rule, and sparse kernels do not automatically make an otherwise identical pretrained model work at new lengths.

PRA relation and experiment. BigBird constructs content-independent shortcuts inside one anonymous token graph. PRA constructs content-dependent edges to explicitly named external objects. Compare a matched sparse-edge budget in which random/global block edges and top- k URI objects expose the same number of keys. Sweep distractor blocks, wrong handles, evidence distance, and repeated queries. Report cold and warm materialization for PRA; otherwise BigBird is unfairly charged only for attention while PRA hides encoding. An instructive hybrid replaces random inter-document edges with URI-restricted routing and retains BigBird within each selected document.

3.4 Performer

Motivation and mechanism. Performer approximates softmax attention with positive random features (FAVOR+), exchanging a $T \times T$ matrix for associative summaries [Choromanski et al., 2021]. If $\exp(q^\top k) \approx \phi(q)^\top \phi(k)$, then

$$\text{Attn}(Q, K, V) \approx D^{-1} \phi(Q) (\phi(K)^\top V), \quad D = \text{diag}(\phi(Q) (\phi(K)^\top \mathbf{1})). \quad (17)$$

For fixed feature count, time and memory are linear in sequence length. Causal execution uses prefix sums of $\phi(k_i) v_i^\top$ and $\phi(k_i)$.

Listing 7: Causal kernel-attention recurrence; phi must be a positive feature map

```
def causal_linear_attention(qf, kf, v, eps=1e-6):
    kv = torch.einsum('...tm,...td->...tmd', kf, v).cumsum(dim=-3)
    kz = kf.cumsum(dim=-2)
    num = torch.einsum('...tm,...tmd->...td', qf, kv)
    den = torch.einsum('...tm,...tm->...t', qf, kz).unsqueeze(-1)
    return num / den.clamp_min(eps)
```



Evidence, failure modes, and lineage. Performer reports image, text, and protein experiments and measures both downstream quality and approximation/efficiency. Its paper trail lies in kernelized linear attention and random-feature estimation. Feature variance, finite-precision instability, causal implementation details, and inability to recover a specific token after destructive aggregation are central failure modes. Linear cost does not mean selective relevance: every token contributes to the statistic.

PRA relation and comparison. Performer approximates computation after eager materialization; PRA changes which objects enter computation. The two are complementary: use linear attention for the local stream or within a resolved memory. A fair experiment crosses exact versus FAVOR+ local attention with all-local versus PRA access, reports approximation error and exact-value recall, and sweeps random-feature count under fixed FLOPs. PRA corruption tests remain necessary because a low approximation error says nothing about whether the correct reference was selected.

4 Category II: Recurrence and Compression

Recurrence changes the temporal boundary: earlier segments leave behind state rather than remaining as raw local tokens. The central distinction from PRA is *write-time retention versus read-time dereferencing*. Recurrent systems decide what history survives as it passes; PRA assumes an addressable backing object can be fetched later.

4.1 Transformer-XL

Motivation and mechanism. Transformer-XL addresses fixed context and segment fragmentation by carrying the previous segment’s hidden states forward as stop-gradient memory and using relative positional attention [Dai et al., 2019]:

$$\tilde{H}_s^{(\ell-1)} = [\text{SG}(H_{s-1}^{(\ell-1)}); H_s^{(\ell-1)}], \quad Q = H_s W_Q, \quad K, V = \tilde{H}_s W_{K,V}. \quad (18)$$

Only current-segment states receive gradients through the recurrent boundary. The memory extends effective context without backpropagating across the entire history; relative positions prevent a cached state from being confused with a current absolute position.

Listing 8: Segment recurrence; detach is the defining optimization boundary

```
def transformer_xl_segment(layer, x, old_memory, memory_len):
    kv_source = torch.cat([old_memory.detach(), x], dim=1)
    y = layer(query=x, key=kv_source, value=kv_source, relative_positions=True)
    new_memory = torch.cat([old_memory, x], dim=1)[: , -memory_len:].detach()
    return y, new_memory
```



Evidence and limitations. The paper evaluates WikiText-103, One Billion Word, Penn Treebank, enwiki8, and text8 using perplexity or bits-per-character, alongside recurrent and Transformer language-model baselines. It reports state-of-the-art results at the time and much faster evaluation than recomputing overlapping windows. Ablations isolate recurrence and relative positions. Memory is chronological, anonymous, fixed-length, and not revisable: evicted information is gone, irrelevant history consumes capacity, and a detached boundary limits credit assignment.

PRA relation and direct test. Transformer-XL answers “what recent state should remain?”; PRA answers “which named remote object should be loaded now?” Compare them on streams with either recency-correlated evidence or remote named evidence. Match resident KV bytes and total encoding FLOPs, sweep distance beyond the recurrent window, and include repeated access to the same old object. A hybrid keeps Transformer-XL memory for recent discourse while letting a URI handle reload evicted or non-contiguous source material.

4.2 Compressive Transformer

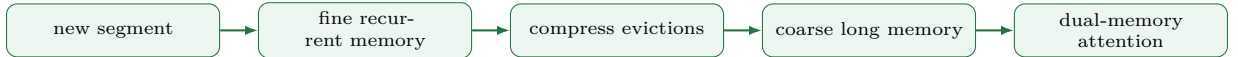
Motivation and mechanism. Compressive Transformer extends Transformer-XL with a second, lower-resolution memory. States evicted from fine memory M^m are transformed by a compressor C before entering compressed memory M^c [Rae et al., 2020]:

$$M_s^c = \text{tail}_{n_c}([M_{s-1}^c; C(E_{s-1})]), \quad (19)$$

where E_{s-1} denotes the fine states being evicted. Attention reads both stores. Pooling, convolution, and usage-based compression trade temporal resolution for longer reach; auxiliary reconstruction losses encourage retained predictive information.

Listing 9: Two-tier chronological memory update

```
def update_memory(fine, compressed, new_states, fine_cap, comp_cap, compress):
    merged = torch.cat([fine, new_states], dim=1)
    evicted, fine = merged[: , :-fine_cap], merged[: , -fine_cap:]
    compressed = torch.cat([compressed, compress(evicted)], dim=1)
    return fine.detach(), compressed[: , -comp_cap:].detach()
```



Evidence and failure modes. The original work studies long-range language modeling, compares compression functions and auxiliary objectives, and uses perplexity plus attention and memory diagnostics. The method’s honest trade-off is rate versus fidelity. Pooling can erase identifiers, superposition can merge distractors, learned compression can optimize average prediction while losing rare facts, and chronological stores cannot directly invalidate a stale semantic object.

PRA relation and experiment. PRA ordinarily reloads an original or cached object; Compressive Transformer obligatorily summarizes passing history. A useful hybrid attaches several materializations to one URI: metadata, summary, compressed latent, and full KV, with uncertainty-driven escalation. Compare fine-only recurrence, dual-memory compression, summary-only PRA, and full-object PRA under equal resident bytes. Sweep compression ratio, rare exact facts, semantic synthesis, and stale-object replacement. Report whether escalation recovers failures and charge full-object encoding in cold-cache trials.

4.3 Recurrent Memory Transformer (RMT)

Motivation and mechanism. RMT inserts a small set of learned memory tokens into every segment. Their output states become the memory-token inputs to the next segment, creating an end-to-end learned bottleneck [Bulatov et al., 2022]. If m_s are incoming memory tokens and x_s current tokens,

$$[\hat{m}_s, H_s, m_{s+1}] = \text{Transformer}([m_s, x_s, m_s^{\text{write}}]). \quad (20)$$

The exact placement varies by implementation, but the conceptual operation is stable: read old slots, process the segment, and write a fixed number of new slots.

Listing 10: RMT segment interface

```
def rmt_step(model, tokens, memory, write_tokens):
    sequence = torch.cat([memory, tokens, write_tokens], dim=1)
    hidden = model(sequence)
    next_memory = hidden[:, -write_tokens.size(1):]
    token_hidden = hidden[:, memory.size(1):-write_tokens.size(1)]
    return token_hidden, next_memory
```



Evidence, lineage, and failures. RMT is evaluated on language modeling and algorithmic/long-sequence tasks; later work demonstrates extrapolation on very long synthetic problems. Baselines include finite-window Transformers and recurrent-memory variants, with loss, task accuracy, and length extrapolation as metrics. The learned bottleneck is simple and differentiable, but it must anticipate future needs at write time. Rare details can be overwritten, slot semantics lack stable provenance, and backpropagation depth or memory curricula strongly affect training.

PRA relation and experiment. RMT learns *what to remember while reading*; PRA learns *what to fetch while answering*. Compare both on matched memory bytes under two datasets: one where future questions are predictable during ingestion and one where they are not. Substitute or reset RMT slots and shuffle PRA references to measure content use. A hybrid stores an RMT summary in the URI metadata tier but preserves the source object for read-time expansion when the summary is insufficient.

4.4 Infini-attention

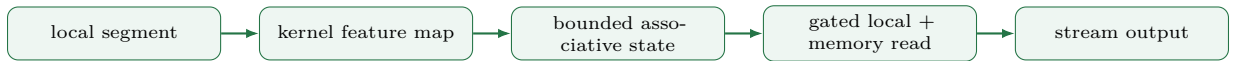
Motivation and mechanism. Infini-attention combines masked local attention with a bounded associative memory inside each block [Munkhdalai et al., 2024]. With positive feature map ϕ , a segment updates

$$M_s = M_{s-1} + \phi(K_s)^\top V_s, \quad z_s = z_{s-1} + \sum_i \phi(k_{s,i}), \quad (21)$$

and reads $\phi(Q_s)M_{s-1}/(\phi(Q_s)z_{s-1})$. A learned gate mixes this compressive read with dot-product attention over the current segment. Memory size is independent of total processed length, which makes streaming execution possible.

Listing 11: Associative-memory read and write

```
def infinif(qf, kf, v, M, z, eps=1e-6):
    read = torch.einsum('btd,bdh->bth', qf, M)
    denom = torch.einsum('btd,bd->bt', qf, z).unsqueeze(-1)
    read = read / denom.clamp_min(eps)
    M = M + torch.einsum('btd,bth->bth', kf, v)
    z = z + kf.sum(dim=1)
    return read, M.detach(), z.detach()
```



Evidence and limitations. The paper evaluates long-context language modeling, one-million-token passkey retrieval, and 500K-token book summarization with 1B- and 8B-scale models. Metrics include perplexity, retrieval accuracy, summarization quality, and memory scaling. Bounded state is the strength and the limitation: many keys superpose, collisions and normalization can obscure exact facts, and there is no explicit object version, authorization boundary, or selective invalidation.

PRA relation and direct comparison. Infini-attention offers constant-size lossy history; PRA offers selective identity with potentially unbounded backing storage and bounded resident working set. Match resident bytes and local window, then sweep number of objects, repeated keys, exact-value queries, semantic summaries, and stale replacements. PRA must include resolution/encoding latency. A hybrid maintains cheap Infini state for the stream while a collision, low-confidence query, or explicit handle faults in a named PRA object.

5 Category III: Differentiable and Episodic External Memory

These methods move state outside the ordinary recurrent or token representation while keeping access near neural computation. They introduce two questions that recur in PRA: who writes the memory, and whether an address denotes a stable object or only a similar vector.

5.1 Memory Networks

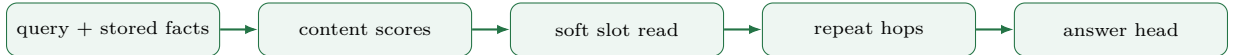
Motivation and mechanism. Memory Networks separate input encoding, memory storage, addressing, and response production [Weston et al., 2015]. Facts become slots m_i ; a query q produces a content read

$$p_i = \text{softmax}_i(q^\top m_i), \quad o = \sum_i p_i c_i, \quad q' = q + o. \quad (22)$$

Repeating the read implements multi-hop reasoning. End-to-end variants learn embeddings from question-answer supervision; earlier variants used stronger supporting-fact labels.

Listing 12: One differentiable memory hop

```
def memory_hop(query, address_slots, content_slots):
    weights = torch.softmax(query @ address_slots.transpose(-1, -2), dim=-1)
    read = weights @ content_slots
    return query + read, weights
```



Evidence, lineage, and failure modes. The paper trail runs from supervised Memory Networks through end-to-end memory networks and key-value memory networks. Evaluations include synthetic bAbI reasoning and question answering, using answer accuracy and supporting-fact retrieval. Multiple hops make compositional access inspectable, but soft attention across a large flat store scales poorly; embeddings can exploit lexical shortcuts; and overwrite, provenance, and slot lifecycle are under-specified for production knowledge.

PRA relation and experiment. Memory Networks establish the learned-read primitive that PRA reuses, but PRA delegates object writes and identity to a resolver/cache. Compare flat learned slots with URI candidate restriction followed by the same within-candidate attention. Hold stored facts and read hops fixed; sweep memory size, paraphrase, distractors, and wrong-object substitutions. Report supporting-fact recall and counterfactual answer change, not attention maps alone. The expected trade-off is open semantic discovery versus cheap exact restriction when a handle is available.

5.2 Differentiable Neural Computer (DNC)

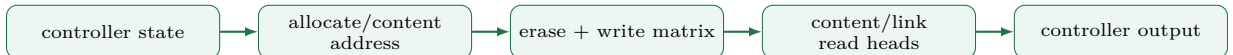
Motivation and mechanism. The DNC couples a neural controller to a read/write matrix with content lookup, temporal links, usage-based allocation, and differentiable erase/add operations [Graves et al., 2016]. A write updates row i as

$$M_t(i) = M_{t-1}(i) \odot (1 - w_t^w(i)e_t) + w_t^w(i)a_t, \quad (23)$$

while read weights mix content addressing with forward/backward traversal of a learned temporal linkage. It therefore supports both associative recall and ordered walks.

Listing 13: Core DNC erase/add write; allocation and link updates omitted

```
def dnc_write(memory, write_weight, erase, add):
    w = write_weight.unsqueeze(-1)
    return memory * (1 - w * erase.unsqueeze(1)) + w * add.unsqueeze(1)
```



Evidence and limitations. DNC experiments emphasize synthetic graph traversal, question answering, and algorithmic tasks, comparing recurrent controllers and earlier memory systems with exact task accuracy or sequence error. The machine can learn elegant algorithm-like behavior, yet many interacting gates make training brittle. Soft access is expensive at large slot counts, learned addresses lack stable external meaning, and catastrophic write errors can corrupt later computation.

PRA relation and experiment. DNC learns both memory management and access; PRA externalizes durable writes, versions, and permissions, narrowing learning to selection and fusion. Compare DNC, read-only DNC, and PRA on a graph task whose nodes have stable IDs plus a variant with IDs hidden. Match active memory bytes and controller parameters. Test node substitution, temporal traversal, stale updates, and multi-tenant isolation. A hybrid can use a small DNC as transient scratch space while PRA supplies immutable, auditable backing objects.

5.3 Memformer

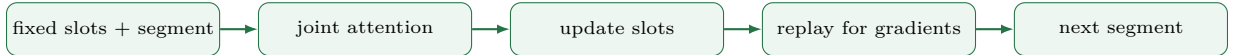
Motivation and mechanism. Memformer carries a fixed number of dynamic memory slots across segments and introduces memory replay backpropagation so training need not retain every old activation [Wu et al., 2020]. A segment attends to slots and local tokens, then a write operation updates the slots:

$$S_{t+1} = \text{Write}(S_t, H_t), \quad |S_t| = N_s \text{ constant.} \quad (24)$$

Replay reconstructs selected memory states during backward computation, trading extra compute for lower activation storage.

Listing 14: Simplified fixed-slot segment update

```
def memformer_step(block, tokens, slots):
    joint = torch.cat([slots, tokens], dim=1)
    hidden = block(joint)
    new_slots = hidden[:, :slots.size(1)]
    token_hidden = hidden[:, slots.size(1):]
    return token_hidden, new_slots
```



Evidence, ablations, and failures. The paper evaluates sequence modeling under long inputs, reporting task quality, training memory, and speed against Transformer and memory-augmented baselines. Slot count, recurrence, and replay are key ablations. Constant space is attractive, but slots are lossy summaries without stable semantic names; useful facts may be overwritten; replay adds complexity; and a slot cannot be independently revoked or refreshed when its source changes.

PRA relation and experiment. Memformer provides a learned working set; PRA adds a named backing store and explicit faults. Use identical segment encoders and match resident slots/KV bytes. Ask questions whose evidence is either predictable during ingestion or selected only after ingestion, and sweep time-to-query. Compare slot reset/substitution with PRA disabled/shuffled controls. A tiered design stores Memformer summaries as cheap URI metadata and faults in full layer-specific memory when summary confidence is low.

5.4 Memorizing Transformer

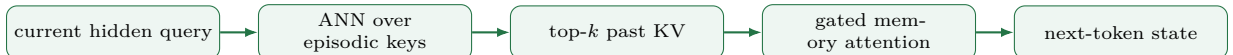
Motivation and mechanism. Memorizing Transformer records past key/value pairs in a large non-differentiable episodic database and retrieves nearest keys for current queries [Wu et al., 2022]. For query q ,

$$\mathcal{I} = \text{kNN}(q, \{k_i\}), \quad o_{\text{mem}} = \text{softmax}(qK_{\mathcal{I}}^T)V_{\mathcal{I}}, \quad (25)$$

which is gated with ordinary attention. The store can span many batches and does not need gradients through nearest-neighbor selection.

Listing 15: Episodic KV retrieval; an ANN index replaces the dense distance

```
def memorizing_read(q, key_db, value_db, k):
    dist = torch.cdist(q.float(), key_db.float())
    idx = dist.topk(k, largest=False).indices
    keys, values = key_db[idx], value_db[idx]
    score = torch.einsum('...d,...kd->...k', q, keys)
    return torch.einsum('...k,...kd->...d', score.softmax(-1), values)
```



Evidence and failure modes. Experiments span language-model datasets and model scales up to billions of parameters, measuring perplexity versus memory size and retrieval configuration. The method demonstrates that latent episodic retrieval can improve a Transformer without differentiating through a huge store. Its risks are near-duplicate memorization, index cost, stale states after model updates, cross-example contamination, and weak source provenance. Similarity can retrieve the right surface continuation while failing compositional evidence use.

PRA relation and experiment. This is a close latent-memory precursor. The main difference is open kNN over anonymous stream states versus two-stage object identity and within-object attention. Compare (a) global episodic kNN, (b) exact URI restriction, (c) semantic object routing then token kNN, and (d) oracle object restriction. Hold total indexed KVs and returned keys fixed. Include near-duplicate continuation, multi-object composition, wrong URI, and stale-model cache conditions, and report source-level recall alongside perplexity.

6 Category IV: Retrieval-Augmented Pretraining and Generation

Retrieval augmentation places a corpus outside model parameters. The decisive axes are whether selection is supervised or latent, whether retrieved material enters as text or hidden state, and whether retrieval occurs once, per chunk, or per generated token.

6.1 kNN-LM

Motivation and mechanism. kNN-LM stores training-context representations paired with observed next tokens. At inference it builds a distance-weighted neighbor distribution and interpolates it with the parametric language model [Khandelwal et al., 2020]:

$$p_{\text{kNN}}(y | q) \propto \sum_{i: y_i=y} \exp(-d(q, k_i)/\tau), \quad (26)$$

$$p(y | q) = \lambda p_{\text{kNN}}(y | q) + (1 - \lambda) p_{\text{LM}}(y | q). \quad (27)$$

No retrieved passage is read compositionally; memory changes the final token distribution.

Listing 16: kNN-LM probability interpolation

```
def knn_lm(log_p_lm, q, keys, next_tokens, k, vocab, lam, tau):
    d, idx = torch.cdist(q, keys).topk(k, largest=False)
    weights = torch.softmax(-d / tau, dim=-1)
    p_knn = torch.zeros(*q.shape[:-1], vocab, device=q.device)
    p_knn.scatter_add_(-1, next_tokens[idx], weights)
    return torch.logaddexp(log_p_lm + math.logip(-lam),
                          p_knn.clamp_min(1e-12).log() + math.log(lam))
```



Evidence, lineage, and limitations. The original work reports perplexity on WikiText-103 and domain-adaptation settings, comparing the base LM and cache/retrieval baselines. Improvements without parameter updates motivate later datastore adaptation, pruning, and efficient kNN work. Storage and ANN latency can be large; interpolation needs calibration; privacy risks follow from training-example memorization; and the mechanism is strongest for continuation similarity rather than multi-fact reasoning.

PRA relation and comparison. kNN-LM fuses at the output distribution; PRA fuses rich memory within hidden computation. Compare them with the same corpus encoder and retrieval budget on near-duplicate prediction versus questions requiring composition. Report perplexity, calibration, exact-match, datastore bytes, and latency. Add shuffled token labels for kNN-LM and shuffled objects for PRA. If PRA only matches kNN-LM on near-copy tasks at higher cost, its richer transport is not justified.

6.2 REALM

Motivation and mechanism. REALM jointly pretrains a dense retriever and a masked language model over an external corpus [Guu et al., 2020]. With latent document z ,

$$p(y | x) = \sum_{z \in \text{TopK}(x)} p_{\eta}(z | x) p_{\theta}(y | x, z), \quad (28)$$

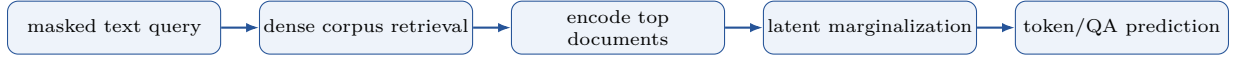
where approximate maximum inner-product search supplies candidates and the language loss provides a learning signal to the retriever. Retrieved text is encoded jointly with the masked input, so the knowledge store can change without retraining all model parameters.

Listing 17: REALM-style latent-document marginalization

```

scores, doc_ids = retriever.topk(query_encoder(masked_input), k)
log_p_doc = scores.log_softmax(-1)
log_p_y = torch.stack([reader(masked_input, corpus[i]) for i in doc_ids], -1)
loss = -torch.logsumexp(log_p_doc + log_p_y, dim=-1).mean()

```



Evidence and failure modes. REALM evaluates open-domain QA and knowledge-intensive pretraining, comparing retrieval-free language models and retrieval pipelines with exact match and retrieval diagnostics. Its important legacy is end-to-end retriever learning from the task objective. Index refresh is expensive, top- k truncation biases the latent sum, corpus snapshots become stale, and retrieval may learn shortcuts when answers are predictable locally.

PRA relation and experiment. PRA should borrow REALM’s trainable selection rather than rely on fixed summary cosine scores. REALM materializes retrieved text in an encoder; PRA can reuse per-object layer KVs and explicit handles. Cross oracle/learned retrieval with text/cross-attention transport on the same corpus. Match retrieved tokens and index queries; evaluate locally sufficient and reference-required subsets, shuffled documents, random handles, warm caches, and index refresh after source updates.

6.3 Retrieval-Augmented Generation (RAG)

Motivation and mechanism. RAG combines a dense retriever with a seq2seq generator. RAG-Sequence holds a document mixture for a whole output; RAG-Token permits the latent document to vary by token [Lewis et al., 2020]:

$$p(y \mid x) = \prod_t \sum_{z \in \text{TopK}(x)} p_\eta(z \mid x) p_\theta(y_t \mid x, z, y_{<t}). \quad (29)$$

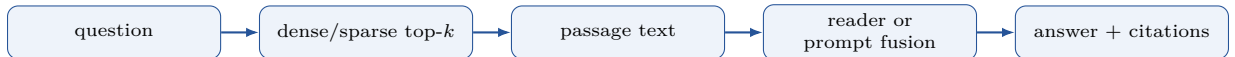
In common systems practice, retrieved passages are concatenated into a prompt rather than exactly marginalized, but the selection/materialization distinction is the same.

Listing 18: Minimal retrieve-then-read RAG pipeline

```

def rag_answer(question, retriever, generator, k):
    hits = retriever.search(question, k=k)
    context = "\n\n".join(f"[{h.id}] {h.text}" for h in hits)
    return generator(f"Sources:\n{context}\nQuestion: {question}\nAnswer:")

```



Evidence, descendants, and failures. RAG evaluates Natural Questions, TriviaQA, WebQuestions, CuratedTREC, MS MARCO-style knowledge tasks, and generation, comparing DPR, closed-book, and retrieval-augmented baselines with exact match and generation/diversity metrics. It seeded extensive work on reranking, query rewriting, corrective and agentic RAG. Failure modes include chunk boundary errors, retrieval miss, distractor sensitivity, prompt-token cost, citation mismatch, and stale indexes. A fluent answer does not prove that retrieved evidence caused the output.

PRA relation and direct comparison. Both keep knowledge external; they differ in timing and materialization. RAG usually chooses text before generation, while PRA can expose named latent objects by layer or step. Use one corpus, retriever, generator backbone, and evidence budget. Compare prompt text, encoder cross-attention, cached object KVs, and iterative retrieval. Report task quality, citation precision/recall, valid/disabled/shuffled effects, cold/warm latency, tokenization/encoding cost, and behavior as queries over the same documents repeat.

6.4 RETRO

Motivation and mechanism. RETRO retrieves neighbors for fixed-size chunks from a massive token database, encodes each neighbor and its continuation, and interleaves chunked cross-attention in the decoder [Borgeaud et al., 2022]. For input chunk x_c and neighbors N_c ,

$$N_c = \text{kNN}(E(x_c), \mathcal{D}), \quad H'_c = H_c + \text{CrossAttn}(H_c, \text{Enc}(N_c)). \quad (30)$$

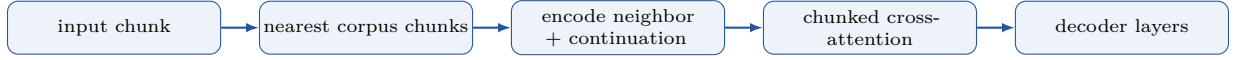
Retrieval happens at chunk boundaries, bringing non-parametric data closer to layer computation than prompt-only RAG.

Listing 19: RETRO-style chunk retrieval and cross-attention

```

for chunk in split(tokens, chunk_len):
    ids = index.search(chunk_encoder(chunk), k=neighbors)
    neighbor_states = encoder(corpus.with_continuations(ids))
    h = decoder_chunk(chunk, retrieved=neighbor_states) # selected layers cross-attend

```



Evidence, ablations, and limitations. RETRO trains models with a database on the order of trillions of tokens and compares perplexity and downstream behavior against larger parametric Transformers. Retrieval neighbors, database scale, cross-attention placement, and corpus overlap are central ablations. Results support non-parametric scaling, but the system inherits corpus/index cost, possible benchmark leakage, chunk-boundary mismatch, and retrieval latency; source identity is incidental rather than an explicit user-facing contract.

PRA relation and experiment. RETRO is one of PRA’s closest architectural ancestors: both transport external representations through cross-attention. PRA adds explicit handles, object boundaries, layer-specific reusable cache identity, progressive admission, and causal reference controls. Under one corpus and backbone, compare no handle, noisy handle, exact handle, and global semantic retrieval; keep returned tokens and PRA layers fixed. Measure selection recall, quality, per-stage latency, cache reuse, source corruption, and multi-object composition. Oracle selection isolates whether PRA’s transport works before router claims.

7 Category V: Attention-Level Retrieval and KV Runtime Management

This category is easy to conflate because all methods manipulate latent states. Unlimiformer performs semantic token selection; PagedAttention virtualizes physical allocation; the four KV-retention methods decide which already-produced stream tokens remain resident. Only the first is a retriever in the usual information-access sense.

7.1 Unlimiformer

Motivation and mechanism. Unlimiformer lets a pretrained encoder-decoder process arbitrarily segmented input, indexes all encoder hidden states, and replaces dense decoder cross-attention with exact attention over MIPS-selected states [Bertsch et al., 2023]. Because the cross-attention score is an inner product, each decoder query can retrieve its largest candidate logits without first forming the full query-key matrix:

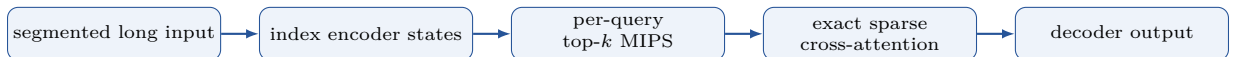
$$\mathcal{I}(q) = \text{TopK}_i(q^\top k_i), \quad o = \text{softmax}(qK_{\mathcal{I}}^\top)V_{\mathcal{I}}. \quad (31)$$

Listing 20: Token-state MIPS followed by exact attention

```

def unlimiformer_cross_attention(q, index, value_store, k):
    scores, ids = index.search(q, k)
    values = value_store[ids]
    return torch.einsum('...k,...kd->...d', scores.softmax(-1), values)

```



Evidence and limitations. The paper evaluates long-input encoder-decoder tasks, especially summarization, against truncated, segmented, and long-context baselines using ROUGE and memory/length measures. It also studies training-free wrapping and fine-tuning. Selection occurs at token granularity and can miss jointly useful low-ranked tokens; the index is tied to the current encoded input; ANN setup and query cost remain; and source provenance must be reconstructed from token offsets.

PRA relation and comparison. Unlimiformer and PRA meet at sparse latent attention. Unlimiformer indexes all token states of the current long input, while PRA first selects named objects and may persist their per-layer KVs. Compare full cross-attention, Unlimiformer token MIPS, PRA object routing plus dense within-object attention, and a hierarchical object-to-token cascade. Match returned KVs, encoder FLOPs, and peak bytes; sweep object size, distractors, repeated queries, and object-level corruption. The cascade tests whether explicit identity improves efficiency without sacrificing token precision.

7.2 PagedAttention

Motivation and mechanism. PagedAttention is a serving-memory design in vLLM, not a semantic attention rule. It partitions a sequence’s KV cache into fixed-size physical blocks addressed through a logical block table, reducing fragmentation and enabling copy-on-write sharing [Kwon et al., 2023]. For logical token position t ,

$$b = \lfloor t/B \rfloor, \quad o = t \bmod B, \quad (K_t, V_t) = \text{blockTable}[b][o]. \quad (32)$$

Listing 21: Logical-to-physical KV lookup

```
def paged_lookup(logical_positions, block_table, blocks, block_size):
    logical_block = logical_positions // block_size
    offset = logical_positions % block_size
    physical_block = block_table[logical_block]
    return blocks[physical_block, offset]
```



Evidence, systems lineage, and failures. vLLM evaluates serving throughput, latency, memory utilization, and scheduling across model sizes and workloads against contemporary inference systems. Block size, batching, sharing, and scheduling are systems variables rather than accuracy ablations. Paging cannot decide which knowledge is useful; small blocks increase metadata, large blocks reintroduce internal fragmentation, and shared physical storage must preserve isolation and copy-on-write semantics.

PRA relation and experiment. The designs compose: PRA supplies semantic object identity and PagedAttention supplies allocation. Map an immutable (u, v, ℓ, m, p) object to reference-counted KV blocks and maintain per-sequence block tables/masks. Compare contiguous and paged PRA caches under cold, warm, and authorized cross-request reuse. Measure bytes allocated, fragmentation, blocks shared, avoided encoding, time to first token, throughput, eviction, and a confidentiality test where another sequence’s private cache must leave the target logits invariant.

7.3 StreamingLLM

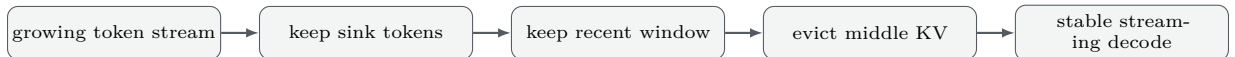
Motivation and mechanism. StreamingLLM observes that autoregressive models often assign disproportionate attention to initial “sink” tokens even when those tokens are not semantically important. Retaining a few initial KV’s plus a rolling recent window allows stable streaming beyond the training context [Xiao et al., 2023]:

$$\mathcal{K}_t = \{0, \dots, s-1\} \cup \{t-w+1, \dots, t\}. \quad (33)$$

Middle-history KV’s are evicted; no semantic retrieval or reconstruction is attempted.

Listing 22: Attention-sink plus rolling-window retention

```
def streaming_keep(length, sinks=4, window=1024):
    first = torch.arange(min(sinks, length))
    recent = torch.arange(max(sinks, length-window), length)
    return torch.unique(torch.cat([first, recent]))
```



Evidence and failure modes. The paper tests long streaming language modeling and generation with LLaMA-style models, comparing sliding windows and cache baselines through perplexity and long-run behavior. Sink tokens repair an attention-distribution pathology; they do not preserve arbitrary facts. Middle-history details are irrecoverable, sink behavior depends on model training, and the method is not a solution for non-chronological external knowledge.

PRA relation and experiment. StreamingLLM manages the local chronological cache; PRA retrieves named backing objects. Combine them by keeping sinks/recent discourse locally and dereferencing remote evidence. Under a fixed total KV budget, compare rolling window, sink+window, PRA, and the hybrid. Sweep query distance, explicit handles, and whether lost facts are still present in the backing store. Charge PRA faults and test shuffled content; test StreamingLLM with sink removal and exact middle-history probes.

7.4 H₂O: Heavy-Hitter Oracle

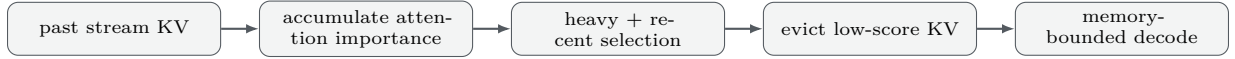
Motivation and mechanism. H₂O treats KV eviction as a dynamic submodular selection problem. It retains recent tokens and “heavy hitters” with large accumulated attention contribution [Zhang et al., 2023]. A simplified score is

$$I_i^{(t)} = I_i^{(t-1)} + \sum_h A_{h,t,i}, \quad \mathcal{K}_t = \text{Recent}_r \cup \text{TopK}_h(I^{(t)}). \quad (34)$$

The policy reacts to observed decoder attention rather than using a static window.

Listing 23: Educational heavy-hitter update

```
def h2o_keep(importance, attn_to_past, recent_ids, heavy_budget):
    importance[:attn_to_past.numel()] += attn_to_past.detach()
    heavy = importance.topk(heavy_budget).indices
    return torch.unique(torch.cat([heavy, recent_ids])), importance
```



Evidence and limitations. H₂O evaluates OPT, LLaMA, and GPT-NeoX across language and generation tasks, measuring quality preservation, cache size, throughput, and latency against inference systems and eviction baselines. Its reported systems gains are conditional on model, hardware, batch, cache fraction, and implementation. Accumulated attention is a lagging proxy, early errors are irreversible, low-attention setup facts can matter later, and raw attention mass is not causal importance.

PRA relation and experiment. H₂O decides which states from one active stream remain; PRA decides which external objects enter. Use H₂O within each selected PRA object or across the resident object working set, but preserve URI/version metadata for every KV. Compare full object KVs, H₂O-compressed objects, object-level eviction, and both tiers. Sweep delayed-use facts, distractors, object count, and warm-cache reuse. Removal/substitution tests should verify that retained heavy hitters, rather than generic branch activation, cause correct answers.

7.5 Scissorhands

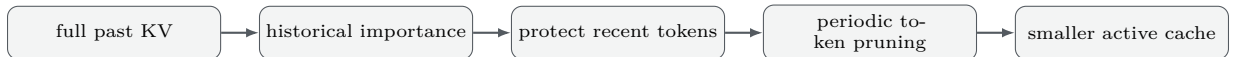
Motivation and mechanism. Scissorhands builds on the “persistence of importance” hypothesis: tokens important in earlier attention tend to remain important later. It maintains importance statistics and discards KVs predicted to stay unimportant [Liu et al., 2023]. At an eviction step,

$$\mathcal{K} = \text{Recent}_r \cup \text{TopK}_{B-r}(\text{score}_{\text{history}}(i)). \quad (35)$$

Policies can be applied per head/layer and periodically rather than at every token.

Listing 24: Periodic history-score pruning

```
def scissorhands(kv, score, recent, budget):
    protected = torch.arange(max(0, len(score)-recent), len(score))
    old = torch.arange(0, max(0, len(score)-recent))
    keep_old = old[score[old].topk(max(0, budget-len(protected))).indices]
    keep = torch.cat([keep_old, protected])
    return kv.index_select(-2, keep), score.index_select(0, keep)
```



Evidence and failure modes. The work evaluates cache reduction across language models/tasks using perplexity or task quality plus memory and throughput. Comparisons with window and heuristic eviction establish the value of history-aware scores. Importance persistence can fail under topic shifts or late references; dropped tokens cannot be reconstructed; per-head policies complicate kernels; and attention history remains a correlational diagnostic.

PRA relation and experiment. Scissorhands compresses already materialized stream history, whereas PRA can reload a stable object after eviction. Compare irreversible token eviction, eviction plus raw backing-text reload, and eviction plus cached PRA object KVs. Match total bytes and include late queries after long distractor gaps. Report reload rate, quality, end-to-end latency, and counterfactual content effects. Object boundaries may provide a cleaner eviction unit than anonymous tokens, but that is a testable hypothesis.

7.6 SnapKV

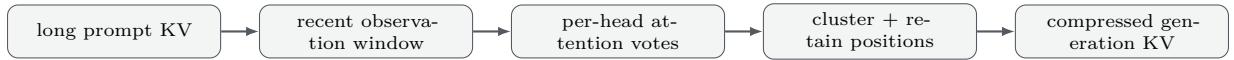
Motivation and mechanism. SnapKV compresses a long prompt’s KV cache before generation by using attention from a recent observation window to identify prompt positions likely to matter during decoding, then pooling nearby selections [Li et al., 2024]. A schematic per-head rule is

$$s_i = \text{pool} \left(\sum_{q \in W_{\text{obs}}} A_{q,i} \right), \quad \mathcal{K} = \text{TopK}_B(s). \quad (36)$$

Unlike online eviction, it takes a “snapshot” of prompt relevance before the answer grows.

Listing 25: Observation-window prompt selection

```
def snapkv_scores(attn, observation, pool=7):
    # attn: [heads, query, key]; exclude generated/recent keys as appropriate
    score = attn[:, -observation:, :].sum(dim=1).unsqueeze(1)
    return F.max_pool1d(score, kernel_size=pool, stride=1, padding=pool//2).squeeze(1)
```



Evidence and limitations. SnapKV evaluates long-context QA/retrieval and generation across instruction-tuned LLMs, measuring task quality, prompt-cache reduction, and speed against eviction/compression baselines. Its main assumption is that the observation window predicts future importance. Later reasoning can change the relevant region; prompt positions lose object/version identity; selection errors are irreversible unless the source remains elsewhere; and head-wise layouts complicate reuse.

PRA relation and experiment. SnapKV is query-aware token compression after eager prompt ingestion; PRA is object admission before or during latent use. Compare full prompt, SnapKV, PRA, and PRA with SnapKV-compressed object KVs. Match resident bytes and include questions whose relevance is evident initially versus revealed after intermediate reasoning. Measure prefill, selection, cold encoding, decode latency, exact evidence retention, and valid/shuffled object effects. A multi-fidelity PRA cache could store a SnapKV snapshot and escalate to full object KVs on low confidence.

8 Category VI: Structural and Hierarchical Indexing

Flat chunk retrieval treats document structure as noise. Hierarchical and graph approaches make sentences, sections, entities, communities, or claims explicit intermediate units. Their advantage is not merely fewer tokens: the index supplies a reasoning topology and a natural provenance coordinate system.

8.1 Hierarchical Attention Networks (HAN)

Motivation and mechanism. HAN mirrors document composition: a bidirectional word encoder and attention pool create sentence vectors, then a sentence encoder and attention pool create a document vector [Yang et al., 2016]. At either level,

$$u_i = \tanh(Wh_i + b), \quad \alpha_i = \frac{\exp(u_i^\top u_c)}{\sum_j \exp(u_j^\top u_c)}, \quad v = \sum_i \alpha_i h_i. \quad (37)$$

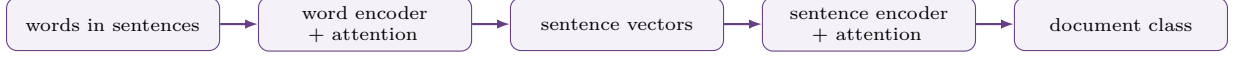
The context vector u_c learns which units matter for classification. Hierarchy reduces a variable document to semantically meaningful intermediate representations.

Listing 26: Reusable hierarchical attention pool

```

class AttnPool(nn.Module):
    def __init__(self, d):
        super().__init__(); self.proj = nn.Linear(d, d); self.context = nn.Parameter(torch.randn(d))
    def forward(self, h, mask=None):
        score = torch.tanh(self.proj(h)) @ self.context
        if mask is not None: score = score.masked_fill(~mask, -torch.inf)
        return torch.einsum('bt,btd->bd', score.softmax(-1), h)

```



Evidence and limitations. The original paper evaluates document classification on Yelp, IMDB, Yahoo Answers, and Amazon review datasets against bag-of-words, convolutional, and recurrent baselines using classification accuracy. Word- and sentence-level attention visualizations are useful diagnostics, not causal explanations. The hierarchy is fixed at two levels, pooling is lossy, bidirectional encoding is not a streaming solution, and the model is designed for one document-level prediction rather than open-ended retrieval.

PRA relation and experiment. HAN supplies the principle that intermediate document units deserve representations. PRA generalizes those units to named sections, tables, records, and code symbols and permits later expansion. Compare flat chunks, HAN-style sentence summaries, and URI hierarchy on classification plus evidence-required QA. Remove or substitute high-attention sentences and selected PRA nodes. A strong hybrid uses HAN pooling to construct cheap summaries while preserving children as resolvable objects.

8.2 RAPTOR

Motivation and mechanism. RAPTOR recursively embeds and clusters leaf chunks, asks a language model to summarize each cluster, and repeats until a tree of increasingly abstract nodes is formed [Sarathi et al., 2024]. With nodes V_0 at the leaves,

$$V_{\ell+1} = \{\text{summarize}(C) : C \in \text{cluster}(\{E(v) : v \in V_\ell\})\}. \quad (38)$$

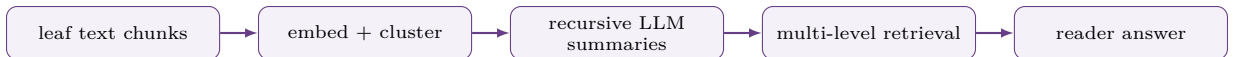
Retrieval can search a collapsed set of all levels or traverse the tree, letting a query choose between detailed evidence and broad synthesis.

Listing 27: Bottom-up RAPTOR index construction

```

level = chunk(document)
tree = [level]
while len(level) > 1:
    groups = cluster(embed(level))
    level = [summarizer(group) for group in groups]
    tree.append(level)
return tree

```



Evidence, paper trail, and failures. RAPTOR reports QA results on QuALITY, NarrativeQA, and QASPER, comparing conventional retrieval and long-context/retrieval baselines with accuracy or task metrics. The paper highlights a 20-point absolute QuALITY gain for its GPT-4 configuration; that number is conditional on its model, tree, prompts, and evaluation. Descendants study dynamic maintenance and cheaper tree construction. Indexing is expensive, abstractive summaries can omit or invent facts, clustering is unstable, and an error at a high node can misroute an entire subtree.

PRA relation and experiment. RAPTOR chooses representations during indexing; PRA chooses when and at which layer a representation becomes resident. Assign immutable URIs to every tree node and let the router escalate from summary to children. Compare flat RAG, collapsed-tree retrieval, explicit tree traversal, and PRA transport using the same tree. Measure answer quality, retrieved source tokens, LLM index cost, expansions, cache reuse, summary corruption, and whether leaf evidence supports generated claims.

8.3 MemWalker

Motivation and mechanism. MemWalker recursively summarizes a long document into a memory tree and has an LLM navigate it by choosing children until sufficient evidence is found [Chen et al., 2023]. At node v_t , a policy reads the query and child summaries:

$$a_t \sim \pi_\theta(a \mid q, s(v_t), \{s(c) : c \in \text{child}(v_t)\}), \quad v_{t+1} = \text{child}(v_t, a_t). \quad (39)$$

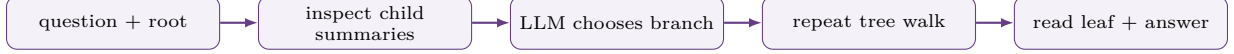
The traversal trace externalizes intermediate access decisions instead of retrieving all nodes in one ranking call.

Listing 28: Bounded tree walk

```

node = root
for depth in range(max_depth):
    action = navigator.choose(question, node.summary,
                             [c.summary for c in node.children])
    if action == "answer": break
    node = node.children[action]
return reader(question, node.source_text)

```



Evidence and failure modes. MemWalker evaluates long-context reasoning/QA against retrieval and long-context baselines, measuring answer quality and traversal behavior. It is closer to an agent than static RAG, but the action space is confined to a prebuilt tree. Serial LLM calls add latency, greedy early choices can make relevant siblings unreachable, summary errors compound, and navigation traces show decisions without proving causal use of the selected leaf.

PRA relation and experiment. Both implement progressive disclosure. MemWalker serializes branch decisions as language-model actions; PRA proposes a latent router inside selected layers. Use the same tree and compare textual navigation, latent top- k child selection, oracle paths, and a hybrid that escalates ambiguous nodes to the agent. Match node-expansion budgets and report latency, depth, backtracking, path recall, valid/shuffled leaf effects, and calibration at the escalation boundary.

8.4 PageIndex

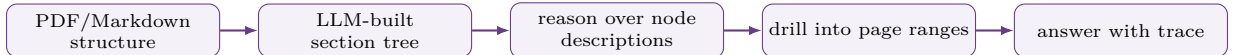
Motivation and mechanism. PageIndex builds a table-of-contents-like tree from natural document sections and uses an LLM to reason over titles, summaries, and page ranges rather than relying on vector similarity [Zhang et al., 2025]. A typical node is (id, title, $[p_s, p_e]$, summary, children); retrieval is a reasoned tree search followed by page/section extraction.

Listing 29: Reasoning-first section traversal

```

frontier = [root]
while budget.remaining() and not evidence_sufficient(frontier, query):
    decision = llm_rank(query, describe(frontier))
    frontier = expand_selected(frontier, decision.node_ids)
pages = merge_ranges([n.page_range for n in frontier])
return answer(query, pdf.extract(pages))

```



Evidence status and reproducibility. PageIndex is an evolving open-source/product system, not a settled peer-reviewed architecture result. This survey pins its inspection target to repository commit [7592163e2a37](#) (mirror snapshot synchronized 15 June 2026) and records the repository’s then-documented default `gpt-4o-2024-11-20`; access date is 3 August 2026. The repository advertises 98.7% FinanceBench accuracy, but this remains a vendor-reported compound-system result. Reproduction must freeze PDFs/questions, parser/OCR, tree-building and answering prompts, model versions, judge, exact-match policy, retries, and dollar/token cost. Natural structure can itself be missing or malformed, and every LLM tree call can introduce nondeterminism.

PRA relation and experiment. PageIndex supplies a strong logical namespace for PRA: node IDs/page ranges become versioned URIs, and selected sections can materialize as text or per-layer KV. Compare vector RAG, PageIndex traversal, PRA latent traversal, and agent escalation on the same tree. Stratify exact lookup, cross-reference, multi-section synthesis, and cross-document reasoning. Report source-page recall, answer/citation quality, tree calls, latency, cost, and invalidation after a PDF revision.

8.5 GraphRAG

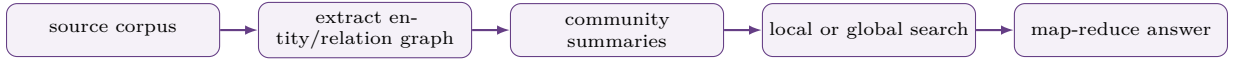
Motivation and mechanism. GraphRAG extracts entities, relations, and claims from a corpus, clusters the resulting graph into communities, and creates community reports [Edge et al., 2024]. Local search expands around query entities; global search maps a question over community reports and reduces partial answers:

$$\hat{y} = \text{reduce}\{\text{answer}(q, R_c) : c \in \text{select}(q, \mathcal{C})\}. \quad (40)$$

The graph exposes corpus-wide themes and multi-hop relationships that may not be similar to the surface query.

Listing 30: GraphRAG global map-reduce sketch

```
reports = select_community_reports(query, graph.communities, token_budget)
partials = [llm.map_answer(query, report) for report in reports]
return llm.reduce_answer(query, partials)
```



Evidence and failure modes. The GraphRAG preprint studies global sensemaking over roughly million-token corpora and reports LLM-judged comprehensiveness and diversity relative to conventional RAG. These are inference-time judgments over an expensive LLM-derived index, so model, prompts, judge, and cost are integral conditions. Extraction errors, entity resolution, hallucinated relations, index cost, stale derived summaries, and judge bias are prominent risks. Local factual QA and global thematic synthesis are distinct tasks.

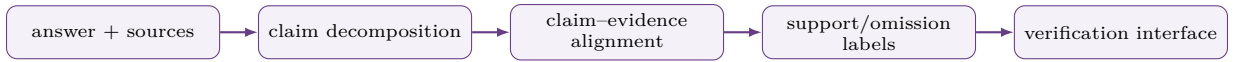
PRA relation and experiment. A PRA URI can address a node, neighborhood, graph query result, or community report. The hard problem becomes dependency-aware invalidation: changing one source may stale many derived KVs. Compare flat RAG, GraphRAG text transport, and graph-node PRA transport on local and global questions. Match source and answer models; report index cost separately from per-query cost, provenance to original spans, stale-update tests, and counterfactual removal of decisive nodes.

8.6 PaperTrail as provenance and HCI

Motivation and mechanism. PaperTrail is not a context architecture. It decomposes an answer and its sources into claims and evidence, then visualizes supported claims, unsupported assertions, and omitted source information [Martin-Boyle et al., 2026]. If C is answer claims and E source evidence units, the interface exposes a relation $R \subseteq C \times E$ plus unsupported $C \setminus \text{dom}(R)$ and uncovered source claims.

Listing 31: Claim–evidence audit pipeline

```
answer_claims = decompose_claims(answer)
source_claims = decompose_claims(sources)
links = align_and_verify(answer_claims, source_claims)
render(answer_claims, source_claims, links,
        unsupported=set(answer_claims)-links.claims())
```



Evidence and limitations. The 2026 work evaluates PaperTrail in a within-subjects study with 26 researchers performing scholarly editing tasks. The interface lowered trust relative to a baseline but did not produce corresponding behavioral change: participants still relied on generated edits when verification was cognitively costly. This is an HCI result about provenance presentation, not evidence for longer context or retrieval quality. Claim decomposition and alignment can themselves be wrong, and more visible evidence can overload rather than assist a user.

PRA relation and experiment. PRA’s URI/version and routing traces can populate a PaperTrail-like view, but attended memory is not causal support. Record selected objects, source spans, cache versions, generated claims, and outputs under removal/substitution. Compare citation-only, route-trace, and counterfactual claim-evidence interfaces. Measure verification accuracy, time, calibrated trust, unsupported-claim detection, omissions, and cognitive load. This positions PaperTrail correctly as provenance/HCI rather than a competitor in the context architecture table.

9 Category VII: Agents with Search and Tool Use

Agents move relevance decisions outside a single forward pass. They can discover unknown sources, reformulate a question after observing evidence, act on the world, and backtrack. Their generality costs serial latency, tokens, tool failures, and a larger audit surface. PRA should be compared as a restricted fast path for repeated read-only access, not as a replacement for open-ended action.

9.1 ReAct

Motivation and mechanism. ReAct interleaves natural-language reasoning with actions and observations, allowing evidence acquisition to change the next reasoning step [Yao et al., 2023]. A trajectory is

$$\tau = (q, r_1, a_1, o_1, r_2, a_2, o_2, \dots, y), \quad (r_t, a_t) \sim \pi_\theta(\cdot \mid q, \tau_{<t}). \quad (41)$$

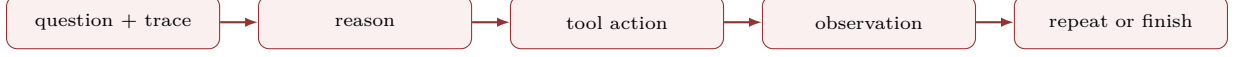
Reasoning maintains a task state; tools ground or alter that state; the policy decides when to stop. Search is only one environment—the paper also studies interactive tasks.

Listing 32: ReAct execution loop with an explicit budget

```

trace = []
for step in range(max_steps):
    thought, action = model.next(question, trace)
    if action.name == "finish": return action.answer, trace
    observation = tools[action.name](**action.arguments)
    trace.append((thought, action, observation))
raise BudgetExceeded(trace)

```



Evidence, descendants, and failures. ReAct evaluates HotpotQA and FEVER plus ALFWorld and WebShop, comparing standard prompting, chain-of-thought, act-only, and ReAct using exact match, factual accuracy, or environment success. Prompt examples, base model, tool interface, and action budget materially condition results. Descendants add planning, reflection, memory, and richer tools. Common failures are reasoning loops, malformed calls, irrelevant searches, premature stopping, prompt injection in observations, and high latency.

PRA relation and experiment. Both progressively acquire context. ReAct remains necessary when sources are unknown, actions have side effects, or explicit planning and backtracking are essential. Compare ReAct search, PRA over a known reference graph, and a tiered policy that tries local context, cached URI memory, structural navigation, then search. Match evidence/action budgets and report answer quality, calls, tokens, latency, provenance, cache reuse, and escalation calibration. Cache successful read-only discoveries as versioned references, never silently replay side-effecting actions.

9.2 Self-Ask with Search

Motivation and mechanism. Self-Ask addresses compositionality by explicitly generating follow-up questions. An external search component answers each subquestion, and the model combines intermediate answers into the final response [Press et al., 2022]:

$$q_t = \text{decompose}(q, a_{<t}), \quad e_t = \text{search}(q_t), \quad y = \text{compose}(q, \{q_t, e_t\}). \quad (42)$$

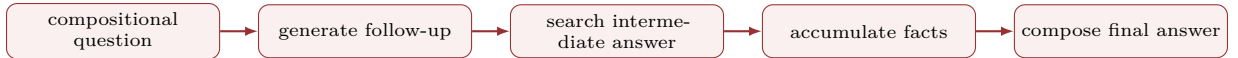
The textual protocol makes the retrieval chain easy to inspect and separates decomposition errors from search errors.

Listing 33: Self-Ask protocol

```

facts = []
while model.needs_followup(question, facts):
    subquestion = model.followup(question, facts)
    evidence = search(subquestion)
    facts.append((subquestion, evidence))
return model.final_answer(question, facts)

```



Evidence and limitations. The work studies multi-hop/compositional question answering and the “compositionality gap”, comparing direct answer, chain-of-thought, and search-assisted decomposition with exact match. Decomposition improves observability, but one wrong subquestion can derail all later hops; intermediate search snippets may be noisy; serial calls cost latency; and a plausible intermediate answer can conceal missing evidence.

PRA relation and experiment. PRA may represent a follow-up target as a generated reference only when a resolver can bind it deterministically; free-form web search is not a URI lookup. Compare text subquestions, latent reference queries, and a hybrid that uses PRA for known namespaces and search otherwise. Use the same search backend and hop budget; measure decomposition accuracy, evidence recall, calls, latency, valid/shuffled reference effects, and failures where the correct source is absent from the advertised graph.

9.3 IRCOT

Motivation and mechanism. IRCOT interleaves retrieval with chain-of-thought so partial reasoning becomes a better query for the next evidence hop [Trivedi et al., 2023]. Rather than retrieve once from the original question,

$$z_t = R(q, r_{<t}, e_{<t}), \quad r_t = G(q, r_{<t}, e_{<t}, z_t), \quad (43)$$

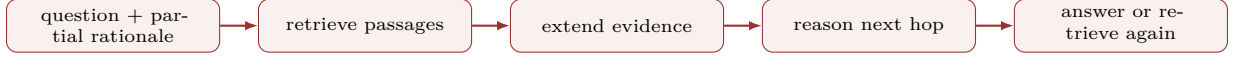
and the cycle continues until an answer or budget limit. Retrieval and reasoning therefore co-evolve.

Listing 34: IRCOT-style retrieval/reasoning alternation

```

state, evidence = question, []
for hop in range(max_hops):
    passages = retriever.search(state, k)
    evidence.extend(passages)
    state, answer = reasoner.step(question, state, passages)
    if answer is not None: return answer, evidence

```



Evidence, paper trail, and failures. IRCOT evaluates multi-step QA including HotpotQA, 2WikiMultiHopQA, MuSiQue, and related datasets, comparing retrieve-then-read, iterative retrieval, and chain-of-thought configurations with exact match/F1 and retrieval metrics. It motivates later iterative and agentic RAG. Reasoning text can introduce false entities that poison later queries, cost grows by hop, stopping is difficult, and answer gains can reflect a stronger prompt rather than a better retriever.

PRA relation and experiment. PRA layers or decoding states could emit successive latent queries, approximating IRCOT without serializing each query. Compare textual query rewriting and latent queries against the same frozen retriever, plus oracle evidence order. Match retrieval calls/candidates and report answer quality, hop recall, latency, and counterfactual evidence effects. The hybrid should expose an auditable object path even if queries are latent; otherwise reduced tokens come at the cost of provenance.

9.4 Toolformer

Motivation and mechanism. Toolformer teaches a language model when and how to call APIs by sampling candidate calls and retaining those that reduce language-model loss [Schick et al., 2023]. For call c inserted at position i , a simplified filter is

$$\Delta(c) = L(x_i: | x_{<i}) - L(x_i: | x_{<i}, c, \text{result}(c)); \quad c \text{ kept if } \Delta(c) > \tau. \quad (44)$$

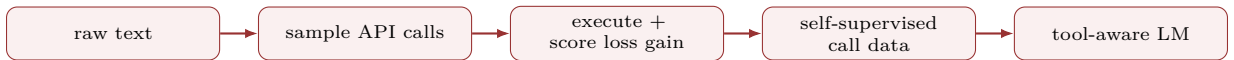
Fine-tuning on retained demonstrations yields discrete call triggering and result integration without dense human call labels.

Listing 35: Loss-improvement filtering for pseudo-labeled calls

```

for position in candidate_positions(text):
    for call in sample_calls(model, text, position):
        result = execute_sandboxed(call)
        gain = loss_without(call) - loss_with(call, result)
        if gain > threshold: training_examples.append(insert(text, call, result))

```



Evidence and limitations. Toolformer evaluates a 6.7B GPT-J model with calculator, QA, search, translation, and calendar tools on zero/few-shot downstream tasks and language modeling, comparing base and tool-aware models. Tool choice and API formatting ablations matter. Pseudo-labels inherit the base model’s blind spots, unsafe or costly calls require sandboxing, tool errors enter the context, and lower token loss does not guarantee factual or task utility.

PRA relation and experiment. Toolformer’s call-retention principle is directly useful for PRA router training: sample candidate object accesses, keep those whose controlled insertion lowers loss, then train a gate with corrupted negatives. Compare heuristic cosine, supervised relevance, and loss-filtered routers under equal trainable parameters. Evaluate selection recall, calibration, task loss, valid/shuffled/irrelevant effects, and access cost. PRA can internalize deterministic reads; arbitrary or side-effecting APIs should remain explicit tool calls.

9.5 WebGPT and browser agents

Motivation and mechanism. WebGPT trains a model to issue search queries, open results, navigate pages, collect quotations, and answer with citations, using demonstrations, behavior cloning, reward modeling, and human feedback [Nakano et al., 2021]. The browser state makes information acquisition a sequential decision process:

$$a_t \sim \pi_\theta(a | s_t), \quad s_{t+1} = \text{browser}(s_t, a_t), \quad y = G(q, \text{quotes}(\tau)). \quad (45)$$

Listing 36: Citation-preserving browser loop

```

state, evidence = browser.start(question), []
for _ in range(action_budget):
    action = policy(state, evidence)
    if action.kind == "quote": evidence.append(browser.quote(action.span))
    elif action.kind == "answer": return compose_with_citations(question, evidence)
    else: state = browser.execute(action)

```



Evidence, descendants, and failures. WebGPT evaluates long-form question answering using human preference, factuality, and citation-related judgments against retrieval and human-written baselines. It established a lineage toward browser-assisted research agents. Results are bound to the web snapshot, search engine, model, demonstrations, reward model, and evaluator. Failures include source-quality errors, selective quotation, citation drift, prompt injection, browsing loops, changing pages, and high serial latency.

PRA relation and experiment. PRA cannot discover an unknown open-web source. It can cache already discovered pages, versions, sections, and evidence for later questions. Compare fresh browser search, cached text RAG, cached PRA KVs, and a policy that searches on cache miss or staleness. Measure answer/citation correctness, source diversity, actions, latency, cache hit and avoided encoding, update detection, and security under malicious page instructions. Provenance must retain the original page/version even when KVs are reused.

9.6 Recursive Language Models and context as environment

Motivation and mechanism. Recursive language-model approaches treat an oversized context as an external environment. A model writes code or issues subcalls that inspect, partition, summarize, and recursively solve pieces instead of consuming the entire source in one forward pass. For context D and budget B ,

$$y = F(q, D, B) = \text{aggregate}\{F(q_j, D_j, B_j)\}_{j=1}^m, \quad \sum_j B_j \leq B. \quad (46)$$

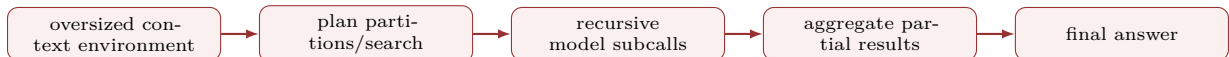
The model controls the decomposition and can adapt it to each question.

Listing 37: Budgeted recursive context program

```

def solve(query, view, budget):
    if view.fits() or budget.depth == 0:
        return model(query, view.read(budget.tokens))
    plan = model.plan_partitions(query, view.metadata(), budget)
    partials = [solve(p.subquery, view.slice(p.range), budget.child(p)) for p in plan]
    return model.aggregate(query, partials)

```

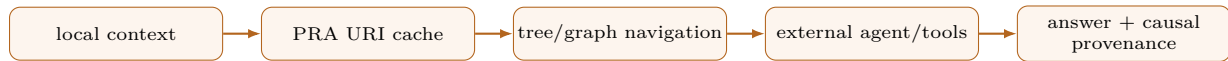


Evidence and failure modes. The family is evaluated on inputs beyond a model’s window, code/document analysis, aggregation, and long-context reasoning, with answer quality, subcall count, tokens, and cost. Baselines should include truncation, long context, RAG, and non-recursive map-reduce. Generality is high, but recursive calls are expensive, partition boundaries can destroy dependencies, summaries compound errors, arbitrary generated code needs sandboxing, and budgets/stopping rules can dominate outcomes.

PRA relation and experiment. PRA can be viewed as a differentiable cached fast path for recurring read operations discovered by an agent or recursive program. Use identical source partitions and compare RLM subcalls, tree navigation, PRA references, and a tiered system that escalates on uncertainty. Match total model FLOPs/tokens and wall-clock budget; report depth, calls, source coverage, provenance, cache reuse, and tasks whose relevant partition is unknown initially. The agent should win on open-ended discovery; PRA must justify itself through repeated low-latency reuse and causal content selection.

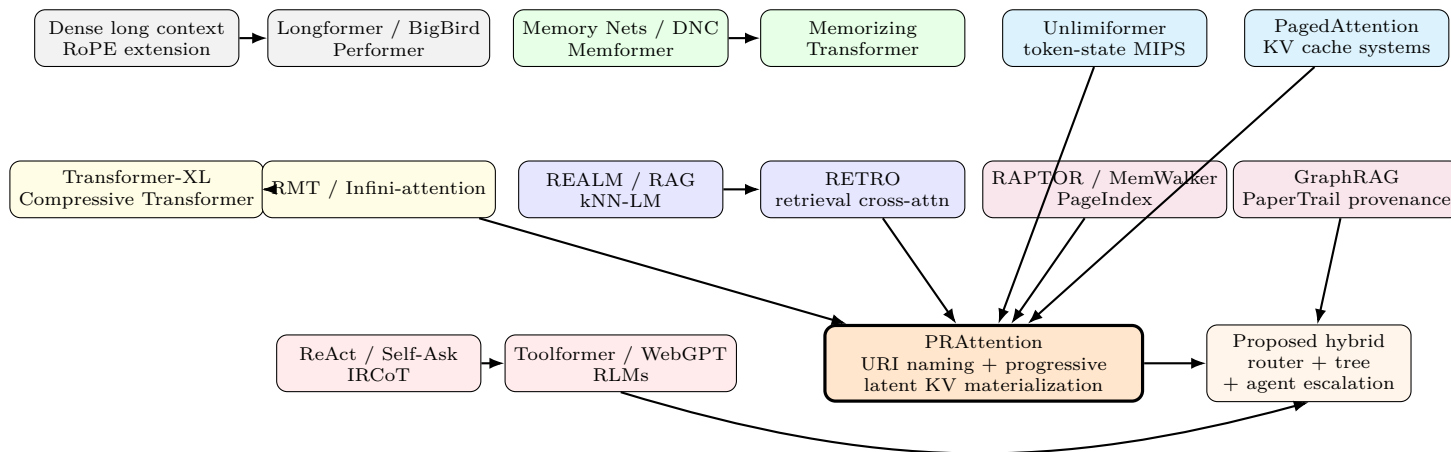
9.7 A tiered agent–PRA design

The categories are best composed as a monotone escalation policy. Local attention handles dense immediate context; a warm URI cache handles repeated named reads; a tree/graph index handles bounded structural search; and a general agent handles discovery or action. Each tier returns both evidence and calibrated confidence. Expensive discoveries may be cached only if the source is immutable or versioned, authorization permits reuse, and the action is read-only. The evaluation must include wrong-confidence costs: unnecessary escalation, silent cache hits on stale data, and premature stopping before an agent search.



10 One-Page Architecture Landscape

Landscape of Long-Context and Retrieval Architectures



Main axes: eager ↔ demand-driven access; flat tokens ↔ named structure; text ↔ latent KV;
static one-shot selection ↔ progressive reasoning; transient context ↔ reusable cache; opaque use ↔ causal provenance.

11 Cross-Architecture Comparison

Family	Store	Selection time	Form	Main strength	Characteristic failure
Dense long context	prompt tokens	before forward pass	token KVs	general interactions	quadratic cost, distraction
Sparse attention	prompt tokens	architecture or task mask	sparse token KVs	long visible sequence	missing sparse paths
Recurrence	prior segments	chronological	hidden/KV state	streaming continuity	lossy or indiscriminate history
Differentiable memory	learned slots/episodes	per layer/step	latent vectors	trainable read/write	optimization and identity
RAG/REALM	document corpus	before or per-token generation	retrieved text encodings	fresh external knowledge	chunking and retrieval miss
RETRO	token chunks	chunk boundary	cross-attention encodings	scale via nonparametric data	index and corpus dependence
KV retrieval	hidden/KV index	attention time	selected states	efficient latent access	provenance and stale cache
Tree/graph index	structural nodes	iterative or one-shot	summaries, spans, or subgraphs	multigranular reasoning	index construction errors
Search agent	open tools/web	iterative decisions	observations or text	maximum flexibility	latency, cost, tool errors
PRA	URI objects and KV caches	layer/step, demand-driven	selected per-layer KV	identity, reuse, progressive access	router training and runtime complexity

12 Adapting Original Experiments to PRA

12.1 A benchmark matrix

No single benchmark distinguishes all mechanisms. A credible program combines:

1. **Language preservation:** WikiText-2/103 or C4 slices before and after memory adaptation.
2. **Exact long-range retrieval:** passkeys, variable needles, UUID/value lookup, and repeated distractors.
3. **Natural long-document understanding:** NarrativeQA, Qasper, QuALITY, GovReport, arXiv/PubMed summarization, and LongBench-style tasks.
4. **Open-domain retrieval:** Natural Questions, TriviaQA, HotpotQA, 2WikiMultiHopQA, MuSiQue.
5. **Structured documents:** FinanceBench, legal filings, technical manuals, code repositories, and table/figure references.
6. **Agentic discovery:** web or corpus search tasks where the correct source is initially unknown.

12.2 Common conditions

For each task, compare: local-only SelfAttention; longer-context SelfAttention; sparse attention; one-shot RAG; iterative RAG; latent token retrieval; PRA with explicit handles; PRA with learned discovery; structural-tree retrieval; and agentic search. Hold corpus, tokenizer, generation model, evidence budget, and evaluation prompts fixed where meaningful.

12.3 PRA-specific causal matrix

Condition	What it tests	Expected interpretation
Valid	complete system	task utility
Disabled branch	any memory contribution	Δ_{use}
Shuffled content	semantic dependence	$\Delta_{content}$
Irrelevant matched length	distractor resistance	router specificity
Random URI, same content	identity dependence	naming shortcut
Same URI, wrong version	cache invalidation	staleness safety
Oracle object	selection upper bound	router headroom
All evidence local	access overhead	efficiency frontier
Agent oracle search	discovery upper bound	unresolved-source headroom

12.4 Metrics beyond task accuracy

Report task quality, retrieval recall, ranking, reference causality, preservation, and systems cost together. Systems measurements must separate resolve time, encode time, cache lookup, routing, attention, generation, and synchronization. Cache hit rate without latency reduction is not useful; latency reduction without answer preservation is not success.

13 Research Lineage and Influence

The field can be read as a sequence of boundary shifts. Memory Networks moved relevant facts outside the recurrent state. Transformer-XL moved context across segments. REALM and RAG moved knowledge outside parameters. RETRO and Memorizing Transformer moved retrieved information closer to hidden-state computation. Unlimiformer recast cross-attention as nearest-neighbor search. PagedAttention made KV storage a virtual-memory systems problem. RAPTOR, MemWalker, and PageIndex restored document hierarchy. ReAct, IRCoT, Toolformer, and WebGPT made information acquisition an action policy. PRA’s proposed step is to make *named reference resolution and reusable latent materialization* a first-class attention capability.

This lineage also tempers novelty claims. Cross-attention over external KV is not new; retrieval during model computation is not new; sparse top- k selection is not new; and caches are not new. PRA’s distinct contribution is the package and interface: URI-level identity, explicit reference syntax, separation of naming and materialization, per-layer cache objects, progressive triggering, and a causal evaluation doctrine. Its scientific value depends on showing that this package creates measurable advantages unavailable from simpler combinations.

14 Design Extensions Derived from the Survey

14.1 Hierarchical Progressive Retrieval Attention

Represent a document as a URI tree. Each node stores a summary KV and child handles. The router first attends to coarse nodes, then expands children only when posterior entropy or expected utility warrants it. This unifies RAPTOR/PageIndex with PRA while preserving traceability.

14.2 Agent-distilled routing

Use a capable search agent to generate trajectories: queries, selected sources, section paths, and evidence spans. Convert these into reference-routing supervision. The deployed PRA model learns a low-latency approximation; the agent remains an escalation path for out-of-index or ambiguous cases.

14.3 Multi-fidelity object caches

For each URI maintain tiers: metadata, summary vector, compressed KV, full KV, and raw source. A learned controller chooses the cheapest tier expected to reduce loss. This connects Compressive Transformer, Infini-attention, PagedAttention, and PRA.

14.4 Reference compilation

Repeated agent/tool operations can be compiled into stable references. A SQL result, code symbol graph, dashboard panel, or document query becomes a versioned object with dependency metadata. PRA then retrieves the result cheaply until invalidated. This is especially relevant to data-analytics agents.

14.5 Causal provenance graph

Record reference selection, cache version, attended spans, generated claims, and counterfactual effects. Join these into a graph compatible with PaperTrail-like claim-evidence views. The strongest evidence edge is not raw attention weight but measured output change under controlled removal or substitution.

15 Discussion

15.1 The real design choice is the access contract

Debates framed as “long context versus RAG” hide the more useful question: what contract exists between computation and external information? Dense context offers anonymous token positions. RAG offers retrieved passages. agents offer observations from named actions. PRA proposes persistent objects with identity, version, representation, and cache lifecycle. Such contracts determine not only quality but governance, sharing, invalidation, debugging, and cost.

15.2 When PRA should not win

PRA is unlikely to dominate when the entire source is short, every token is densely relevant, hardware kernels make dense attention cheap, or the source is unknown and requires open-ended discovery. It also adds complexity where a simple one-shot retriever is adequate. Negative results in these regimes would clarify the architecture rather than invalidate it.

15.3 When PRA has a plausible advantage

The strongest regime has large, repeatedly used, naturally addressable objects; sparse relevance; many queries over shared sources; stable or versioned content; and tasks where relevance becomes clearer only after intermediate processing. Examples include technical documentation, analytics catalogs, code repositories, longitudinal records, enterprise knowledge, and embodied-agent maps.

15.4 Training difficulty

A new memory branch can be ignored, overused, or used as a generic adapter. Near-zero gate initialization, frozen-base staged training, auxiliary selection losses, corrupted-reference negatives, and curriculum by distance are sensible. However, explicit supervision may bias the model toward the indexer’s notion of relevance. End-to-end loss and causal ablations remain necessary.

15.5 Object identity versus semantic similarity

Semantic retrieval answers “what looks relevant?” Explicit addressing answers “which object is being referred to?” Real systems need both. An explicit handle can restrict search to an object namespace; semantic routing can select parts within it; an agent can discover a missing object. Treating one mechanism as universally sufficient is a category error.

15.6 Impact on agents

PRA does not eliminate the agent loop. It can internalize routine read-only actions, reduce repeated tokenization and tool latency, and provide a memory substrate for agent discoveries. The agent retains planning, source discovery, side effects, and exception handling. The long-term architecture is likely layered rather than exclusive.

16 Conclusion

Long-context modeling is not one problem. It is a family of access problems spanning compute graphs, state persistence, external memory, retrieval, indexing, runtime allocation, provenance, and action. The seven categories reviewed here form a continuum from eager anonymous tokens to progressive named actions. PRA occupies a strategically interesting middle point: more structured and reusable than flat long context or one-shot RAG, but potentially cheaper and more differentiable than general search agents.

The current evidence establishes implementation feasibility, ordinary-language compatibility under approximate parameter matching, and preservation under a frozen reference-path adaptation strategy. It does not establish content-selective reference use: the fixed three-seed follow-up leaves the selector near chance, valid references barely outperform shuffled or irrelevant controls, and oracle content does not materially improve loss. The most valuable next experiments are therefore a trainable router and reference-required natural tasks, followed by five-seed replication, hierarchical and agentic baselines, and synchronized measurements of latency, memory, encoding, and cache reuse. If those experiments succeed, PRA could become a useful abstraction for compiling repeated referential work into a model-native memory interface. If they fail, the same experimental program will still clarify where dense context, RAG, structural navigation, and agentic search should meet.

Online Resources

PRA source manuscripts used for this tutorial were supplied by the author. Live links for external works are embedded in the bibliography through DOI or arXiv URLs. The PageIndex discussion refers to its public repository and documentation; claims should be independently reproduced.

References

- Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020. doi:10.18653/v1/2020.emnlp-main.263. URL <https://aclanthology.org/2020.emnlp-main.263/>.
- Amanda Bertsch, Uri Alon, Graham Neubig, and Matthew R. Gormley. Unlimiformer: Long-range transformers with unlimited length input. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2305.01625>.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning*, 2022. URL <https://proceedings.mlr.press/v162/borgeaud22a.html>.
- Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer, 2022. URL <https://arxiv.org/abs/2207.06881>.
- Howard Chen, Ram Pasunuru, Jason Weston, Asli Celikyilmaz, Gheorghe Comanici, and Thibault Févry. MemWalker: Walking through long-context memory with an LLM, 2023. URL <https://arxiv.org/abs/2310.05029>.
- Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations*, 2021. URL <https://arxiv.org/abs/2009.14794>.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019. doi:10.18653/v1/P19-1285. URL <https://aclanthology.org/P19-1285/>.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitan, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization, 2024. URL <https://arxiv.org/abs/2404.16130>.

- Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Danihelka, Agnieszka Grabska-Barwińska, Sergio Gómez Colmenarejo, Edward Grefenstette, Tiago Ramalho, John Agapiou, Adrià Puigdomènech Badia, Karl Moritz Hermann, Yori Zwols, Georg Ostrovski, Adam Cain, Helen King, Christopher Summerfield, Phil Blunsom, Koray Kavukcuoglu, and Demis Hassabis. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538:471–476, 2016. doi:[10.1038/nature20101](https://doi.org/10.1038/nature20101). URL <https://doi.org/10.1038/nature20101>.
- Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*, 2020. URL <https://proceedings.mlr.press/v119/guu20a.html>.
- Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations*, 2020. URL <https://arxiv.org/abs/1911.00172>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023. doi:[10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165). URL <https://doi.org/10.1145/3600006.3613165>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2404.14469>.
- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2305.17118>.
- Anna Martin-Boyle, Cara A. C. Leckey, Martha C. Brown, and Harmanpreet Kaur. PaperTrail: A claim-evidence interface for grounding provenance in LLM-based scholarly Q&A. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, 2026. URL <https://arxiv.org/abs/2602.21045>.
- Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention, 2024. URL <https://arxiv.org/abs/2404.07143>.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. WebGPT: Browser-assisted question-answering with human feedback, 2021. URL <https://arxiv.org/abs/2112.09332>.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models, 2022. URL <https://arxiv.org/abs/2210.03350>.
- Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020. URL <https://arxiv.org/abs/1911.05507>.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2401.18059>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2302.04761>.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023. doi:[10.18653/v1/2023.acl-long.557](https://doi.org/10.18653/v1/2023.acl-long.557). URL <https://aclanthology.org/2023.acl-long.557/>.
- Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1410.3916>.

- Qingyang Wu, Zhenzhong Lan, Kun Qian, Jing Gu, Alborz Geramifard, and Zhou Yu. Memformer: A memory-augmented transformer for sequence modeling, 2020. URL <https://arxiv.org/abs/2010.06891>.
- Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2203.08913>.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2023. URL <https://arxiv.org/abs/2309.17453>.
- Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of NAACL-HLT*, 2016. doi:10.18653/v1/N16-1174. URL <https://aclanthology.org/N16-1174/>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2007.14062>.
- Mingtian Zhang, Yu Tang, and PageIndex Team. PageIndex: Next-generation vectorless, reasoning-based RAG. PageIndex blog and open-source repository, 2025. URL <https://github.com/VectifyAI/PageIndex/commit/7592163e2a376b3917181fff9ac1858dc5daa2c6>. Repository commit 7592163e2a376b3917181fff9ac1858dc5daa2c6; accessed 3 August 2026.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O : Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.14048>.